VidAI

Where Traditions Meet Technology.

Final Year Project

Team Members

- Faiza Alam Warraich (01-131222-016)
- Asad E Bukhari (01-131222-045)

Supervisor

- Dr. Kashif Sultan

Bahria University

Department of Software Engineering

Islamabad, Pakistan

1st December, 2025

Abstract

VidAI is an AI-powered wedding planning platform designed to simplify and automate the end-to-end wedding management process for users in Pakistan. The system provides a unified digital solution that replaces traditional, fragmented planning methods by integrating vendor discovery, service comparison, booking management, budget planning, and digital invitation design into a single application. Leveraging intelligent algorithms and AI microservices, VidAI delivers personalized vendor recommendations, optimized budget allocations, and a smart conversational assistant for guiding users throughout their planning journey.

This document presents the complete Software Requirements Specification (SRS) and Software Design Specification (SDS) for VidAI, detailing the system's functional and non-functional requirements, architectural design, data modelling, interface definitions, and component-level interactions. It serves as a comprehensive technical reference for developers, supervisors, evaluators, and future maintainers involved in the system's development and evolution. The abstract provides a high-level overview of VidAI's purpose, scope, and design objectives to enable a quick understanding of the system's core functionality and architectural vision.

Table of Contents

Department of Software Engineering 1
Islamabad, Pakistan 1
1st December, 2025 1
Software Requirements Specification 6
1. Introduction 6
1.1 Purpose 6
1.2 Document Convention 6
1.3 Project Scope 7
1.4 References 8
2. Overall Description 8
2.1 Product Perspective 8
2.2 Product Functions 10
A. User Functions 11
B. AI Subsystem Functions 11
C. Vendor Functions 11
D. Admin Functions 11
2.3 User Classes and Characteristics 11
2.3.1 End Users (Bride/Groom/Family Members) 11
2.3.2 Vendors (Service Providers) 12
2.3.3 Admin (System Administrators) 12
2.4 Operating Environment 13
2.5 Design and Implementation Constraints 16
2.6 User Documentation 18
2.7 Assumptions and Dependencies 19
A. Assumptions 19
B. Dependencies 20
3. External Interface Requirements 21
3.1 User Interfaces 21
3.2 Hardware Interfaces 24
3.3 Software Interfaces 30
3.3.1 Mobile & Web Client Interfaces 30
3.3.2 Backend API Service Interface 31
3.3.3 AI Microservice Interfaces 31
3.3.4 Database Interface (MongoDB Atlas) 32
3.3.5 Third-Party Service Interfaces 32

3.3.6 Shared Data and Implementation Constraints 33
3.4 Communications Interfaces 34
4. System Features 37
4.1 User Registration & Authentication 37
4.2 User Login 38
4.3 Vendor Search & Filtering 39
4.4 View Vendor Details 39
4.5 Create Booking Request 39
4.6 Vendor Responds to Booking 40
4.7 Budget Planning 40
4.8 AI Recommendations 41
4.9 Digital Invitation Generator 41
4.10 AI Chatbot 42
4.11 Vendor Service Management 42
4.12 Admin Vendor Verification 43
4.13 Notification Delivery 43
4.14 Payment Processing 44
5. Nonfunctional Requirements 44
5.1 Performance Requirements 44
5.2 Safety Requirements 45
5.3 Security Requirements 45
5.4 Software Quality Attributes 46
5.5 Business Rules 47
6. Other Requirements 48
6.1 Database Requirements 48
6.2 Internationalization & Localization Requirements 48
6.3 Legal & Regulatory Requirements 49
6.4 Reuse Objectives 49
6.5 Logging & Monitoring Requirements 50
6.6 Deployment Requirements 50
6.7 Future Enhancement Requirements (Non-Binding) 51
Software Design Specification 52
1. Introduction 52
1.1 Purpose 52
1.2 Document Conventions 53
2. Software Architecture 56

2.1 Architectural Patterns and Styles 56
2.2 High-Level Architectural Overview 57
2.3 Architectural Elements / Modules / Components and Responsibilities 59
2.4 Responsibilities Summary 62
2.5 Architecture Diagrams 63
2.5.1 Logical Architecture (Component Diagram) 63
2.5.2 Physical Architecture (Deployment Diagram) 63
3. Detailed Design 64
3.1 Logical View 64
3.1.1 UML Class Diagram 64
3.2 Dynamic View 65
3.2.1 Sequence Diagrams 65
3.2.2 Activity Diagrams 68
3.3 Deployment View 71
3.3.1 Deployment Diagram 71
3.4 Development View 71
3.4.1 Component Diagram 71
3.4.2 WBS 72
3.5 Data Model 74
3.5.1 ER Diagram 74
3.6 User Interface Design 74
3.6.1 Mobile Application 75
3.6.2 Vendor Web Portal 83
3.6.3 Admin Web Portal 86

Software Requirements Specification

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete set of software requirements for **VidAI – AI-Powered Wedding Planning Application**. It specifies the functional, non-functional, interface, and design constraints required to develop the system.

The purpose of this document is to provide a clear, detailed, and shared understanding of what the VidAI system must accomplish for all stakeholders, including developers, project supervisors, evaluators, vendors, and end-users.

This SRS covers the entire VidAI system, including the mobile application, backend service, vendor management module, booking system, AI-based recommendation engine, budget planner, and digital invitation generator. No subsystem is excluded in this version; the document represents the full scope planned for the FYP implementation.

1.2 Document Convention

This SRS follows the structure and terminology recommended by the **IEEE-830** Software Requirements Specification standard. The following conventions and formatting rules are used throughout the document:

- **Requirement Identification:**

  All functional requirements are labelled as **FR-x**, non-functional requirements as **NFR-x**, and use cases as **UC-x** for traceability.

- **Priority Levels:**

  Each requirement explicitly states its priority using the **MoSCoW model**:

  - **Must Have (M)** – essential for system operation
  - **Should Have (S)** – important but not critical
  - **Could Have (C)** – optional enhancements
  - **Won't Have (W)** – excluded from the current release

Priority is **not inherited**; each requirement includes its own priority designation.

- **Typography:**
  - **Bold text** indicates definitions, key terms, or section headers.
  - *Italic text* highlights notes or assumptions.
  - Monospaced text denotes code, sample API calls, data formats, and system messages.
- **Diagrams and Models:**

UML and architectural diagrams follow **IEEE/ISO 24765** notations unless otherwise stated. Mermaid and PlantUML syntax may be used for rendering diagrams in digital versions.

- **Units and Naming Conventions:**
  - All currency values are in **PKR (Rs)** unless specified.
  - Dates follow the format **DD-MM-YYYY**.
  - Time durations are specified in **seconds (s), minutes (min), or hours (h)**.
  - API payloads use **JSON** unless stated otherwise.

These conventions ensure clarity, uniformity, and ease of reference for all stakeholders interacting with this SRS.

1.3 Project Scope

VidAI is an AI-powered wedding planning ecosystem consisting of a **mobile application for end users** (brides, grooms, and families) and a **web application for vendors and administrators**. The system provides a unified digital platform to simplify and streamline wedding planning activities in Pakistan by enabling users to discover verified vendors, compare services, manage bookings, plan budgets, create digital invitations, and receive personalized recommendations powered by artificial intelligence.

The mobile application (React Native) serves as the primary interface for end users, offering vendor discovery, budget planning, AI recommendations, and booking management. The web application (React) is dedicated to vendors and administrators, enabling vendors to manage profiles, service packages, and booking requests, while administrators oversee vendor verification, system governance, and overall platform health.

VidAI reduces the complexity, time, and manual effort traditionally associated with wedding planning by integrating intelligent decision support and digital management tools into a centralized environment. Key benefits include reduced planning duration, improved cost control, enhanced transparency, minimized vendor miscommunication, and digital transformation of traditional wedding workflows.

VidAI supports several strategic objectives, including:

- Digitizing the Pakistani wedding planning ecosystem by bridging the gap between users and verified vendors.
- Enhancing decision-making through AI-based recommendations aligned with preferences, locations, and budgets.
- Improving vendor reach and operational efficiency via a dedicated vendor management web portal.
- Supporting sustainability by enabling paperless, customizable digital invitation creation.

This SRS defines the functional, non-functional, architectural, and interoperability requirements of VidAI across its **mobile, web, backend, AI microservices, and database**

layers, serving as a unified reference for developers, evaluators, and stakeholders throughout the project lifecycle.

1.4 References

The following documents, standards, and online resources were consulted or referenced in preparing this Software Requirements Specification (SRS) for **VidAI – AI-Powered Wedding Planning Application**:

1. **Chandrasiri, W. H. P. (2021). *Wedding Organizer –*** *Wedding Planning Web Application* (Doctoral dissertation). University of Colombo School of Computing. Retrieved from: https://dl.ucsc.cmb.ac.lk/jspui/handle/123456789/4343
2. **Gupta, K. (n.d.). *Intelligent Event Planning*** *- A Framework for ML-Enhanced Budgeting and Vendor Selection on Mobile Platforms*. Retrieved from: https://www.researchgate.net/publication/390742055_Intelligent_Event_Planning_A_Framework_for_ML-Enhanced_Budgeting_and_Vendor_Selection_on_Mobile_Platforms
3. **Sugathapala, D. B. T. M. (2021). *My Wedima*** – *Wedding Planning Web Application* (Doctoral dissertation). University of Colombo School of Computing. Retrieved from: https://dl.ucsc.cmb.ac.lk/jspui/handle/123456789/4338
4. **Sawalmeh, R. A., & Barakat, R. (2023). The Big Day** - Wedding Planner (Doctoral dissertation, An-Najah National University). Retrieved from: https://repository.najah.edu/bitstreams/121dcd52-6e96-481b-8c5c-940fb2d5ed5a/download
5. **React Native Documentation** – *React Native Official Guide*, Meta Platforms, 2024. Available at: https://reactnative.dev
6. **MongoDB Atlas Documentation** – *Cloud Database Service Documentation*, MongoDB Inc., 2024. Available at: https://www.mongodb.com/docs/
7. **Figma Design Guidelines** – *User Interface and Prototyping Documentation*, Figma Inc., 2024. Available at: https://www.figma.com
8. **VidAI Project Proposal Document**, Asad & Faiza, FYP Proposal Submission, Department of Software Engineering, 2025.

These references provide foundational guidelines, technical specifications, and domain context used in developing the requirements described in this SRS.

2. Overall Description

2.1 Product Perspective

VidAI is a new, self-contained, multi-platform software product designed to function as a comprehensive digital wedding planning ecosystem. It does not replace any existing enterprise system but fills a major gap in Pakistan's fragmented wedding planning landscape,

where users typically rely on informal referrals, social media pages, and unstructured communication channels such as WhatsApp or phone calls.

VidAI is composed of three primary subsystems:

1. **Mobile Application (React Native)** — used exclusively by end users (brides, grooms, and families) for discovering vendors, managing bookings, planning budgets, generating invitations, and interacting with the AI assistant.
2. **Web Application (React)** — used by vendors and administrators.
    - Vendors manage business profiles, service packages, availability, and booking requests.
    - Administrators handle vendor verification, system monitoring, and platform governance.
3. **Backend & AI Infrastructure**
    - A centralized **Node.js/Express backend** acts as the core API gateway and business logic layer for both the mobile and web clients.
    - Dedicated **FastAPI-based AI microservices** handle intelligent features such as vendor recommendation, budget optimization, and chatbot interactions.

These subsystems communicate through secure REST APIs and operate on a unified cloud-hosted data layer powered by **MongoDB Atlas**, ensuring consistent, real-time data access across all user roles and platforms.

VidAI also integrates with several external services, including payment gateways, SMS/email providers, push notification services (FCM), cloud storage platforms, and optional AI-driven media generation APIs to enhance personalization and automation.

Within the broader wedding planning workflow, VidAI serves as the digital orchestrator that centralizes vendor management, improves transparency, reduces manual coordination, and provides AI-enhanced decision support. The system adopts a modular, scalable architectural style, allowing independent evolution of the mobile app, web app, backend services, and AI components while maintaining seamless interoperability.

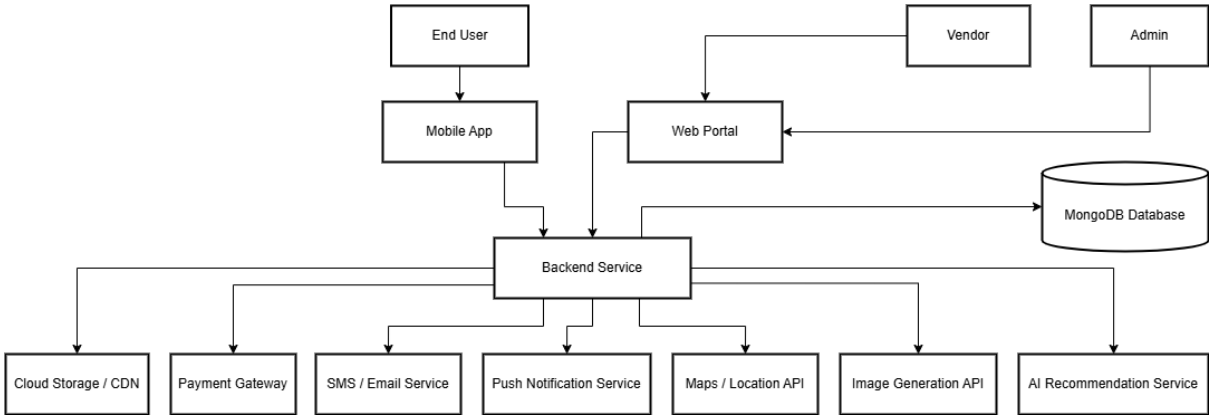


Figure 2.1 — System Architecture Diagram

2.2 Product Functions

The VidAI system provides a structured set of functions that support customers, vendors, admins, and the AI Engine in performing wedding-planning related activities. The major product functions, grouped by actor, are summarized below:

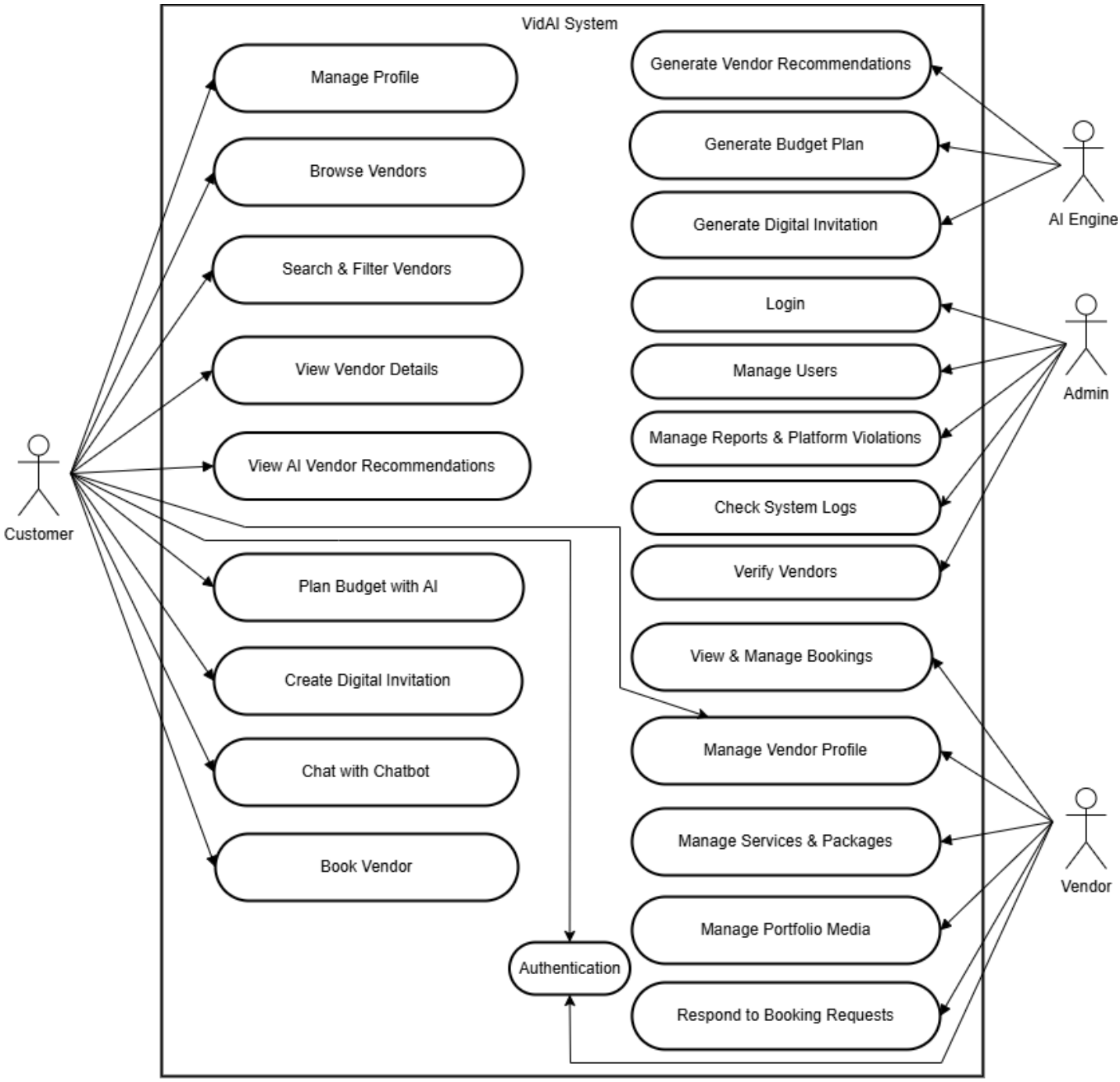


Figure 2.2 - Use Case Diagram

A. User Functions

- **Browse Services & Vendors** – explore wedding vendors and service providers.
- **Filter & Search by Preferences** – narrow down vendors based on budget, location, and service category.
- **Book Vendors & Services** – request vendor bookings and track booking status.
- **Create Digital Invitation Cards** – design and generate wedding invitation cards.
- **Budget Tracker** – monitor expenses and manage planned vs. actual spending.
- **AI Budget Planner** – receive automated budget allocation and optimization.
- **Receive AI Recommendations** – get vendor suggestions, plans, and insights from the AI engine.
- **Chat with Chatbot** – interact with an AI-powered chatbot for information, guidance, and support.

B. AI Subsystem Functions

- **Budget Analysis** – analyze user budget and spending constraints.
- **Generate Plan** – create a personalized wedding plan based on user preferences.
- **Generate Recommendations** – provide ranked, personalized vendor suggestions.
- **NLP & Response Generation** – interpret user messages and generate chatbot responses.

C. Vendor Functions

- **List Services & Packages** – add or update service descriptions, pricing, availability, and offerings.
- **Manage Bookings** – view, accept, or reject booking requests.

D. Admin Functions

- **Manage Users & Vendors** – approve vendors, manage user accounts, and maintain system integrity.
- **Moderation** – editing/removing content.

2.3 User Classes and Characteristics

The VidAI system supports multiple user classes, each with distinct roles, privileges, and functional needs. These classes are differentiated based on system access level, frequency of use, domain expertise, and interaction with system functionalities. The system now supports **two client platforms**—a **mobile app for end users** and a **web application for vendors and administrators**—and user classes are mapped accordingly.

2.3.1 End Users (Bride/Groom/Family Members)

Access Platform: Mobile App (React Native)

Description:
Primary users who utilize the mobile application to plan their wedding.

Characteristics:

- Typically, non-technical users
- Use the application frequently during wedding planning (medium-to-high frequency)
- Require intuitive UI/UX and clear navigation
- Use the majority of system functions, including search, bookings, AI recommendations, budgeting, and digital invitations
- Moderate digital literacy

Importance Level: Critical – The system is primarily designed for this user class.

2.3.2 Vendors (Service Providers)

Access Platform: Web Application (React)

Description:
Businesses or individuals offering wedding-related services (photographers, venues, caterers, decorators, etc.).

Characteristics:

- Moderate digital literacy (varies depending on business type)
- Use functions related to service listing, portfolio management, pricing updates, and booking management
- Log in occasionally or moderately depending on business volume
- Require efficient tools for updating availability and responding to booking requests

Importance Level: High – They provide the supply side of the marketplace, enabling core platform functionality.

2.3.3 Admin (System Administrators)

Access Platform: Web Application (Admin Panel)

Description:
Internal users responsible for managing platform integrity, verifying vendors, moderating content, and ensuring operational stability.

Characteristics:

- High technical knowledge
- Access to protected administrative tools and dashboards
- Uses the system at a professional level
- Needs clear visibility into platform activity, bookings, vendor actions, and system logs
- Uses the system periodically or as needed

Importance Level: Moderate – Essential for governance but not the primary audience.

4. AI Subsystem (Automated Actor)

Access Platform: Backend-facing Service (FastAPI)

Description:
An internal computational subsystem responsible for generating recommendations, analyzing budgets, processing invitations, and providing chatbot responses.

Characteristics:

- Not a human user; functions autonomously
- Interacts with user-facing features through <<includes>> dependency
- Requires structured data inputs and produces deterministic or probabilistic outputs
- Supports all intelligent components of the application

Importance Level: Critical – Powers key differentiating features such as AI planning, recommendations, and chatbot intelligence.

Summary of Importance

User Class	Importance	Reason
End User	Critical	Primary target audience; all core features revolve around them
Vendors	High	Enable marketplace operations and booking workflows
Admin	Moderate	Required for oversight, vendor verification, and system health
AI Subsystem	Critical	Backend intelligence that differentiates VidAI from ordinary platforms

2.4 Operating Environment

VidAI operates in a cloud-enabled, multi-platform environment consisting of a **mobile application**, a **web application**, a **centralized backend**, and **AI microservices**. The system must function reliably across various hardware, software, and network configurations as described below.

1. Mobile Application Environment (End Users – React Native)

Supported Platforms:

- Android smartphones (Android 8.0 Oreo and above)
- iOS devices (iOS 13 and above)

Hardware Requirements:

- Minimum 2 GB RAM
- Stable internet connection (3G, 4G, LTE, or Wi-Fi)
- Touch-enabled device

Client Framework:

- **React Native** (cross-platform mobile development)
- Compatible with Android Studio & Xcode build environments

2. Web Application Environment (Vendors & Admin – React)

Supported Browsers:

- Google Chrome (latest version)
- Microsoft Edge
- Mozilla Firefox
- Safari (for macOS)

Client Technology:

- React.js (SPA architecture)
- Responsive UI framework (Material UI, Tailwind, or equivalent)

Hardware Requirements:

- Any modern desktop/laptop
- Minimum 4 GB RAM
- Stable internet connection

3. Backend Server Environment (Node.js API Layer)

Hosting:

- Cloud-based hosting (AWS, Azure, GCP, Render, Railway, or DigitalOcean)

Operating System:

- Linux-based servers (Ubuntu 20.04+ recommended)

Server-Side Stack:

- **Node.js (v18+)**
- **Express.js** web framework
- RESTful API endpoints
- JWT-based authentication
- NGINX reverse proxy for load balancing & routing

Dependencies and Tools:

- Node package manager (npm/yarn)
- PM2 or Docker for process/container management
- Logging frameworks (Winston/Morgan)

4. AI Subsystem Environment (FastAPI Microservices)

AI Hosting Architecture:

- Python-based microservices using **FastAPI**
- Deployed using Docker containers or serverless platforms
- Communicate with backend through secure internal API calls

AI Libraries and Tools:

- scikit-learn, TensorFlow, or PyTorch (as applicable)
- NLP libraries: spaCy, Transformers (HuggingFace)
- Recommendation and clustering modules
- Budget optimization algorithms

AI Responsibilities:

- Vendor recommendation engine
- Budget planner
- Chatbot / NLP model
- AI-based personalization features

5. Database Environment

Database System:

- **MongoDB Atlas** (cloud-managed NoSQL database)

Supported Features:

- Document storage (BSON format)
- Automatic sharding and replication
- Indexing for high-performance search
- Built-in security: encryption at rest & in transit
- Automated backups and snapshot restores

Access Pattern:

- Shared database used by both mobile and web applications
- CRUD operations via Mongoose ORM (Node.js)

6. External Integration Environment

VidAI integrates with various external services to offer enhanced functionality:

Payment Gateway:

- Easypaisa, JazzCash, Stripe, or other secure payment APIs

Notification Services:

- SMS: Twilio or local SMS APIs
- Email: SendGrid, Mailgun, or SES

Push Notification Environment:

- Firebase Cloud Messaging (FCM) for Android & iOS

Map & Location Services:

- Google Maps & Places API for geolocation, filtering, and route visualization

Cloud Storage Services:

- Cloudinary or AWS S3 for storing images, portfolio media, and invitation assets

Optional AI Enhancements:

- Image-generation API for digital invitation templates (Stable Diffusion, DALL·E, etc.)

7. Network Requirements

- Reliable internet connection required across all modules
- End-to-end **HTTPS-secured** communication
- Minimum bandwidth: **1–2 Mbps** for standard usage
- For media-heavy uploads (vendor galleries), 3–5 Mbps recommended

2.5 Design and Implementation Constraints

The development of VidAI is subject to several design, technical, regulatory, and operational constraints that guide and limit the choices available to the development team. These constraints are outlined below.

1. Platform and Technology Constraints

- The **mobile application must be developed using React Native** to ensure cross-platform compatibility across Android and iOS. This restricts the use of native-only development frameworks.
- The **vendor and admin applications must be developed as a web-based platform using React**, ensuring accessibility across modern browsers.
- The **central backend must be implemented using Node.js (Express.js)**, defining the programming language, runtime environment, and server architecture.
- All intelligent features (recommendation engine, budget planner, chatbot) must be implemented as **Python-based microservices using FastAPI**, requiring interoperability between Node.js and Python services.
- The system must use **MongoDB Atlas** as the primary cloud database, restricting the use of relational schemas.

2. Hardware and Device Constraints

- The mobile app must run efficiently on mid-range Android devices commonly used in Pakistan (2–4 GB RAM), limiting heavy animations, large image assets, or complex on-device AI models.
- The system depends on the user having a stable internet connection; **offline functionality is limited or unavailable**.
- Vendor and admin access is limited to devices supporting modern browsers (Chrome, Edge, Firefox, Safari).

3. External System Dependencies

VidAI relies on multiple third-party services for core functionality:

- **Payment Gateways:** JazzCash, Easypaisa, Stripe (where applicable)
- **Push Notifications:** Firebase Cloud Messaging (FCM)
- **SMS/Email Services:** Twilio, SendGrid, or equivalent

- **Maps/Location Services:** Google Maps / Places API
- **Cloud Storage:** Cloudinary or AWS S3 for vendor portfolios and invitation assets
- **AI Media Services (Optional):** APIs for card/image generation

Any downtime, throttling, or policy limitations of these services directly impacts system reliability and feature availability.

4. Security and Privacy Constraints

- All communication between clients, backend, AI services, and databases must use **HTTPS/TLS encryption**.
- User data and vendor documentation must be stored securely to prevent unauthorized access.
- Although Pakistan lacks formal GDPR-equivalent regulations, the system must uphold ethical standards for data protection, including secure password storage, encrypted tokens, and restricted data access.
- Payment-related data must follow the security standards required by payment service providers.

5. Regulatory and Legal Constraints

- Online payments must comply with **local banking rules**, gateway-specific policies, and digital payment guidelines.
- Vendor verification must follow local business identity norms (e.g., CNIC, business registration documents).
- Use of Google Maps, SMS gateways, and cloud services must comply with their respective terms of service and usage limits.

6. System Performance Constraints

- AI responses—including recommendations, chatbot outputs, and budget analyses—must be generated within **2–5 seconds** to maintain a smooth user experience.
- The system's ability to handle concurrent users is limited by the hosting resources available for an FYP and may not scale to enterprise-level loads.
- Image uploads (vendor portfolios, invitation assets) are constrained by network speed and cloud storage API rate limits.

7. Team and Project Limitations

- The development team consists of **two members**, limiting the complexity and scope of features that can be implemented within the academic timeline.
- Advanced machine learning models must be simplified into lightweight algorithms due to limited training data, computational resources, and development time.

- Certain enterprise-level features such as real-time chat, analytics dashboards, or large-scale recommendation systems may need to be minimized or implemented in a basic form.

8. Development Standards and Conventions

- All services must follow **REST API standards** for structured communication.
- The UI/UX for both mobile and web must follow **Material Design or similar standardized design guidelines**, ensuring consistency and usability.
- Code must follow industry-standard practices including modular architecture, linting rules, proper error handling, and version control via Git.
- Documentation standards must be adhered to for maintainability, including API documentation, code comments, and structured project documentation.

2.6 User Documentation

The VidAI system will be delivered with user support materials to ensure that all user classes—End Users, Vendors, and Admins—can effectively operate the application across both mobile and web platforms. The following user documentation components will be provided:

1. User Manual (Digital – Mobile App)

- A complete guide covering installation, account creation, navigation, vendor search, booking processes, budget tools, AI features, and troubleshooting.
- Delivered as a PDF document and accessible through the mobile app's Help section.

2. In-App Tutorials (Mobile App)

- Interactive onboarding screens guiding new users through core features such as browsing vendors, using filters, making bookings, and generating invitations.
- Step-by-step tooltips displayed during first-time use.

3. Online Help Center (Mobile & Web)

- Hosted on a web-based platform and accessible from both the mobile app and vendor/admin web portal.
- Contains FAQs, how-to articles, and short video clips.
- Updated regularly as new features are added.

4. Vendor Web Portal Guide (Web App)

- Dedicated documentation for vendors explaining how to:
  - Create and manage business profiles
  - Add services and packages
  - Upload portfolio media
  - View and respond to booking requests
  - Delivered digitally as a PDF and accessible from the vendor dashboard.

5. Admin Operations Guide (Web App – Admin Panel)

- Internal documentation for administrators covering vendor verification procedures, system monitoring, user management, and data audits.
- Delivered in PDF format and restricted to admin access only.

6. API Documentation (Internal Use)

- REST API specification generated using Swagger/OpenAPI for both the Node.js backend and FastAPI AI services.
- Assists developers and future maintainers in understanding endpoints, request structures, and data models.

Documentation Format Standards

- All documentation will be delivered in PDF format, adhering to consistent formatting guidelines such as font hierarchy, labeled images, diagrams, and standardized styles.
- In-app guidance will follow the UI/UX standards of **React Native (mobile app)** and **React (web portal)**.

2.7 Assumptions and Dependencies

The following assumptions and dependencies influence the requirements defined in this SRS. These factors are presumed to be true for the successful development, deployment, and operation of the VidAI system. If any of these assumptions change or prove incorrect, the system's requirements, timeline, or feasibility may be significantly affected.

A. Assumptions

1. Users have access to smartphones (End Users only)

It is assumed that end users will own Android or iOS smartphones capable of running the VidAI mobile application.

2. Vendors and Admins have access to modern web browsers

It is assumed that vendors and administrators will use desktop or laptop devices with updated browsers (Chrome, Edge, Firefox, Safari) to access the VidAI web portal.

3. Reliable internet availability

The system assumes that all user classes have stable 3G/4G/LTE/Wi-Fi connectivity for real-time interactions such as bookings, AI processing, notifications, and data updates.

4. Vendors provide accurate and updated information

The system assumes that vendors will correctly list service details, prices, availability, and portfolio information.

5. Vendor participation is sufficient for platform value

The platform's usefulness depends on an adequate number of vendors joining and maintaining active profiles.

6. Third-party APIs remain available and functional

It is assumed that external services such as payment gateways, SMS/email APIs, Firebase Cloud Messaging, Google Maps, and cloud storage remain operational.

7. Dataset availability for AI features

It is assumed that sufficient sample data (vendor attributes, price ranges, user preferences) will be available for experimenting with AI-based recommendation and planning models.

8. Users understand basic mobile and web navigation patterns

The design assumes average digital literacy and familiarity with common UX conventions such as forms, filters, menus, and interactive components.

9. System will not be required to support offline operations

Offline functionality is not required beyond lightweight cached data such as recently viewed vendors.

B. Dependencies

1. Dependency on Payment Gateway Services

Online booking confirmations and prepaid transactions rely on third-party payment systems (JazzCash, Easypaisa, Stripe, etc.).

2. Dependency on Notification Services

SMS, email, and push notifications depend on providers such as Firebase Cloud Messaging (FCM), SendGrid, and local SMS gateways.

3. Dependency on Google Maps API

Location-based filtering, vendor distance calculation, and map rendering require access to Google Maps or similar services.

4. Dependency on Cloud Hosting Providers

The availability and uptime of backend APIs, web apps, mobile services, AI microservices, and the MongoDB Atlas database depend on cloud hosting platforms such as AWS, Azure, or GCP.

5. Dependency on AI Libraries and Frameworks

Chatbot responses, recommendation models, clustering algorithms, and budget planners rely on Python libraries such as TensorFlow, PyTorch, scikit-learn, and NLP frameworks.

6. Dependency on External Data Sources

AI modules may rely on third-party or sample datasets (e.g., market price trends, vendor categories) for improved model performance.

7. Dependency on Team Availability and Skillset

Development progress is dependent on the availability, workload capacity, and technical expertise of the two-member team (Asad and Faiza).

8. Dependency on Device and Browser Capabilities

Performance and UI consistency depend on the specifications of user devices and compatibility of the browser used by vendors/admins. Older devices or outdated browsers may degrade the experience.

3. External Interface Requirements

3.1 User Interfaces

The VidAI system provides intuitive and user-friendly interfaces across both **mobile** and **web** platforms. The **mobile app** is designed for end users (brides, grooms, families), while the **web portal** is designed for vendors and administrators. This section defines the logical characteristics, interface expectations, and UI standards for all user-facing components.

A. General User Interface Standards (Applicable to All Platforms)

1. Consistency

All interfaces must maintain consistent:

- Layout structure and spacing
- Color themes and typography
- Iconography and button styles
- Component hierarchy following Material Design or platform-appropriate conventions

2. Typography & Color Scheme

- Clean, readable sans-serif fonts (e.g., Inter, Roboto)
- Brand-aligned primary, secondary, and accent colors
- WCAG-compliant color contrast

3. Standard Buttons & Actions

- Clear primary/secondary action buttons
- Help/“?” icon for critical forms
- Consistent form validation indicators and error messages

4. Error Message Standards

- Errors must be short, clear, actionable
- Inline validation beneath fields
- Critical failures shown in modal dialogs with retry options

5. Loading & Feedback

- Skeleton screens or spinners for long operations
- Snackbars/toasts for non-blocking notifications

B. Mobile App Interfaces (End Users – React Native)

1. Navigation Standards

- Bottom navigation bar for primary modules:
    - Home, Vendors, Bookings, Budget, Profile
- Top navigation bar with title/back button
- Swipe gestures allowed between tabs (where appropriate)

2. Authentication Interfaces

- Login screen: Email/phone login, password or OTP
- Sign-up screen: Name, email/phone, password
- Onboarding carousel: Skip option included

3. Home & Dashboard Interfaces

- Quick search bar
- AI recommendations
- Top vendors/services
- Upcoming bookings summary

4. Vendor Marketplace Interfaces

- Vendor list screen with card layout
- Filter screen (category, price range, location, rating, availability)
- Vendor detail screen:
    - About section
    - Services & pricing
    - Gallery
    - Book Now & Shortlist buttons

5. Booking Management Interfaces

- Booking request form (date picker, notes)
- Booking status timeline: Pending → Approved/Rejected
- Booking history

6. Budget Planning Interfaces

- Overview of allocations, remaining budget
- AI Budget Planner:
    - Total budget input
    - “Generate Plan” button
- Graphical charts (bar/pie)

7. Digital Invitation Interfaces

- Template gallery
- Editor (text inputs, colors, background selection)
- Preview & export to PNG/JPEG

8. Chatbot Interface

- Chat-style UI (bubbles, timestamp)
- AI quick reply suggestions
- Search vendors through chat
- Error fallback message when AI service fails

C. Vendor Web Portal Interfaces (React Web App)

1. Vendor Dashboard

- Overview of bookings, pending requests, profile completion status

2. Service & Profile Management

- Add/update:
    - Business information
    - Services and pricing
    - Availability calendar
- Upload portfolio images
- Manage visibility status

3. Booking Management

- View booking requests
- Accept/reject bookings
- Chat/contact user (if applicable)
- Booking history logs

4. Analytics (Basic)

- Monthly booking summary
- Profile views (optional)

D. Admin Web Panel Interfaces (React Web App)

1. Vendor Verification

- View submitted vendor documents
- Approve/reject vendor registrations

2. User & Vendor Management

- Block/unblock accounts
- Reset credentials
- Edit essential metadata (if required)

3. Reports Dashboard

- Basic analytics:
    - Total users
    - Total vendors
    - Total bookings
    - Platform activity logs

4. System Monitoring

- Error logs
- AI service status
- API health indicators (optional)

E. Accessibility Requirements (All Platforms)

- Support for large font mode
- Minimum tap/click target: 44×44 px
- Screen-reader compatible labels
- Adequate spacing for touch interactions
- Color-blind friendly palette where applicable

F. Sample Logical UI Flow (User-End)

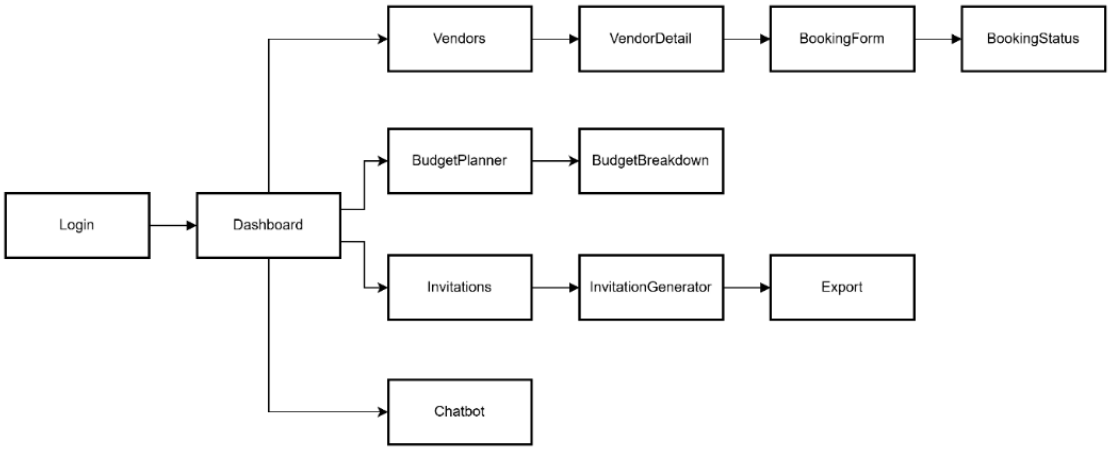


Figure 3.1 - User-End Logical Flow Diagram

3.2 Hardware Interfaces

VidAI interacts with multiple internal and external software components, including mobile and web client environments, backend services, AI microservices, databases, and third-party APIs.

This section describes these interfaces, the data exchanged, and the communication methods used.

A. Mobile Operating System Interfaces (React Native Mobile App)

Supported OS:

1. Android OS (Version 8.0+)
2. iOS (Version 13+)

Purpose:

To execute the VidAI mobile application and provide access to device-level services.

Interfaces Provided:

- Networking stack (HTTP/HTTPS)
- Local secure storage (keychain/secure enclave)
- Location services
- Camera and media gallery access
- Push notification APIs

Data Exchanged:

- Authentication data
- User profiles
- Vendor listings and search results
- Booking data and notifications
- AI recommendations
- Cached images and templates

B. Web Browser Interfaces (Vendor & Admin Web App)

Supported Browsers:

- Chrome, Edge, Firefox, Safari (latest versions)

Interfaces Provided:

- HTTPS communication stack
- File upload APIs for portfolio images
- Web storage (session/local storage) for auth tokens
- Rendering of React components

Data Exchanged:

- Vendor service details
- Pricing, availability, and schedules

- Admin verification data
- Dashboard analytics

C. Backend API Service Interface (Node.js / Express)

Technology:

- Node.js 18+
- Express.js Web Framework
- RESTful API over HTTPS

Purpose:

Central application backend that handles authentication, users, vendors, bookings, budgeting, dashboards, and integrates with external APIs + AI microservices.

Messages Received (Incoming):

- Login/Signup requests
- Vendor search & filter requests
- Booking creation & update requests
- Budget inputs
- Invitation customization data
- Vendor profile & service updates
- Admin verification actions

Messages Returned (Outgoing):

- JWT authentication tokens
- Vendor data & search results
- Booking confirmations & status updates
- Budget analysis results
- AI-generated recommendations
- Invitation template URLs
- Admin dashboards & reports

Shared Data Structures:

- User profiles
- Vendor records & services
- Booking data
- Budget plans
- Templates and media metadata

D. Database Interface (MongoDB Atlas)

Database:

- MongoDB Atlas (Cloud NoSQL)

Driver:

- Mongoose (Node.js backend)
- PyMongo (AI microservices, if needed)

Purpose:

To store all persistent system data.

Data Stored:

- User credentials & profiles
- Vendor details, packages, availability
- Bookings, payments, notifications
- Invitation templates
- AI model metadata & preference vectors

Communication:

- Secure TLS-encrypted MongoDB protocol
- No direct client access—only backend & AI modules communicate with DB

E. AI Microservice Interface (FastAPI Services)

Technology:

- Python 3.10+
- FastAPI (containerized or server-hosted)
- Internal API calls with JSON payloads

Libraries:

- TensorFlow / PyTorch
- scikit-learn
- HuggingFace Transformers
- spaCy / NLP libraries

AI Submodules:

1. Recommendation Engine
2. Budget Planner
3. Chatbot/NLP Handler
4. Smart Wedding Plan Generator

Incoming Data:

- User preferences
- Budget inputs
- Search history & interaction logs
- Vendor categories
- Chat messages (NLP)

Outgoing Data:

- Ranked vendor recommendations
- Budget distribution plans
- Chatbot replies
- Smart planning suggestions

Communication:

- Node.js backend ↔ FastAPI AI using HTTPS internal REST endpoints

F. Third-Party External Software Interfaces

1. Payment Gateway APIs (Easypaisa, JazzCash, Stripe)

Purpose:

- Handle online payments, deposits, and vendor payouts

Data Exchanged:

- Incoming: Payment initiation, amount, transaction metadata
- Outgoing: Payment success/failure status, transaction ID

2. Notification Services

- SMS: Twilio or local telecom APIs
- Email: SendGrid / Mailgun
- Push Notifications: Firebase Cloud Messaging (FCM)

Outgoing Data:

- OTP codes
- Booking confirmations
- System alerts
- Promotional messages

3. Google Maps & Geo APIs

Purpose:

- Vendor geolocation

- Distance computations
- Map rendering

Data Exchanged:

- Incoming: User coordinates, radius, filters
- Outgoing: Place details, routes, distances

4. Image Generation / Template APIs (Optional)

Examples: Stability AI, DALL·E

Purpose:

- Generate invitation card templates

Data Exchanged:

- Incoming: Style/theme prompt
- Outgoing: Image URL or base64 data

5. Cloud Storage / CDN (Cloudinary / AWS S3)

Purpose:

- Store media files (vendor portfolios, invitations, gallery items)

Data Exchanged:

- Incoming: Media uploads
- Outgoing: Secure asset URLs

G. Shared Data Across Components

Component	Shared Data	Interface Method
Mobile App ↔ Backend	User actions, bookings, preferences	REST API (JSON)
Web App ↔ Backend	Vendor/admin data, uploads, approvals	REST API (JSON)
Backend ↔ Database	Full system data	Mongoose / PyMongo
Backend ↔ AI Service	Preference vectors, budget data	Internal API (JSON)
Backend ↔ Payment API	Payment tokens, logs	HTTPS
Backend ↔ Notification API	Messages, contact info	HTTPS
Backend ↔ Maps API	Location queries & coordinates	HTTPS
Backend ↔ Cloud Storage	Media upload/download	HTTPS

Data Sharing Constraint:

- All shared data must use **JSON over HTTPS**.
- **No direct database access** is allowed from mobile or web clients.

H. API Documentation Standard

All VidAI endpoints will follow **OpenAPI/Swagger** specifications for clarity and maintainability.

3.3 Software Interfaces

VidAI interacts with several internal and external software components, including mobile and web clients, backend APIs, AI microservices, databases, and third-party services. This section describes these interfaces and the data exchanged between them.

3.3.1 Mobile & Web Client Interfaces

Components

- React Native Mobile App (End Users)
- React Web Portal (Vendors, Admins)
- Platforms:
    - Android 8.0+
    - iOS 13+
    - Modern browsers (Chrome, Edge, Firefox)

Interface Type

- RESTful HTTP/HTTPS requests to the Backend API.
- JSON is used as the standard message format.

Incoming Messages (from Clients to Backend)

- Authentication requests (login, registration, logout tokens).
- Profile updates (user, vendor, admin).
- Vendor search and filter parameters (category, budget, location, rating).
- Booking actions (create, update, cancel requests).
- Budget planner inputs (total budget, event count, preferences).
- Invitation customization data (text, style choices, template selection).
- Chatbot user messages and context.

Outgoing Messages (Backend to Clients)

- Authentication tokens (JWT), success/failure codes, error messages.
- Vendor lists, vendor details, packages, reviews, availability.
- Booking confirmations, status updates, history.
- AI-generated recommendations, budget breakdowns, and plan summaries.

- Invitation template URLs and saved design data.
- System notifications, alerts, and validation errors.

3.3.2 Backend API Service Interface

Technology

- Node.js (v18+) with Express.js (or similar framework).
- Exposes versioned REST APIs (e.g., /api/v1/...).

Responsibilities

- Authentication, authorization, and session/token validation.
- Orchestration of business logic (vendors, bookings, budgeting, invitations).
- Communication with AI microservices, MongoDB Atlas, and third-party APIs.

Data Format & Protocol

- JSON over HTTPS (TLS 1.2+).
- Stateless request–response model.

Key Service Groups

- /auth/* – login, registration, password reset.
- /users/* – profile management.
- /vendors/* – vendor CRUD, listing, search, filter.
- /bookings/* – create/update/cancel bookings.
- /budget/* – send/receive budget data to/from AI planner.
- /invitation/* – manage templates, user designs, and public URLs.
- /admin/* – moderation, verification, analytics.

3.3.3 AI Microservice Interfaces

Technology

- FastAPI (Python 3.10+), containerized (Docker).
- Libraries: scikit-learn / TensorFlow / PyTorch / NLP frameworks (as needed).

Interface Type

- Internal REST APIs, typically only accessible from the Backend API.
- JSON request/response payloads over HTTPS.

AI Service Endpoints (Examples)

- POST /ai/recommend/vendors
    - Input: user preferences, budget, location, history.
    - Output: ranked vendor list with scores.

- POST /ai/plan/budget
    - Input: total budget, event types, constraints.
    - Output: category-wise allocation and textual explanation.
- POST /ai/chat
    - Input: user query text, conversation context.
    - Output: chatbot response text and optional suggested actions (e.g., open vendor list).

Shared Data

- User preference vectors, historical bookings (as features).
- Aggregated vendor metadata (categories, price ranges).
- AI logs for improvement and debugging (non-sensitive).

3.3.4 Database Interface (MongoDB Atlas)

Technology

- MongoDB Atlas (cloud-hosted NoSQL DB).
- Accessed from backend using Mongoose (Node.js ODM).

Interface Type

- Driver-based connection over TLS-encrypted MongoDB protocol.
- No direct database access from clients or AI services.

Main Collections

- users – user, vendor, admin accounts, roles.
- vendors – vendor profiles, services, pricing, locations.
- bookings – booking records, statuses, payment refs.
- invitations – templates, user-generated designs, file refs.
- ai_logs / preferences – optional AI-related metadata.

Data Sharing Rules

- Backend is the *only* component that directly interacts with the database.
- AI services and clients receive data only through backend APIs.
- Sensitive fields (passwords, tokens) are stored hashed/secured.

3.3.5 Third-Party Service Interfaces

VidAI uses several integrated commercial components:

1. **Payment Gateway APIs**
    - Examples: JazzCash, EasyPaisa, Stripe.

    - Protocol: HTTPS REST API, JSON payloads.
    - Data sent: amount, currency (PKR), order ID, callback URL, customer reference.
    - Data received: transaction ID, status, error codes, payment receipt metadata.
2. **Notification Services**
    - SMS gateway (local providers, Twilio).
    - Email services (e.g., SendGrid).
    - Push notifications (Firebase Cloud Messaging).
    - Data sent: recipient identifiers, message content, templates, metadata.
3. **Maps & Location Services**
    - Google Maps / Places API.
    - Data sent: latitude/longitude, search radius, location queries.
    - Data received: formatted addresses, coordinates, distance, place details.
4. **Cloud Storage / CDN**
    - Cloudinary, AWS S3 or similar.
    - Data sent: image files for vendor portfolios and invitation cards.
    - Data received: secure file URLs, metadata (dimensions, format).
5. **Optional Image/Template AI APIs**
    - Used for generating background designs or invitation templates.
    - Input: textual prompt, style options.
    - Output: image URLs or base64-encoded images.

3.3.6 Shared Data and Implementation Constraints

- All software components share data exclusively via **JSON over HTTPS**; no shared memory or global data areas are used.
- The **backend API** acts as the central integration point; clients and AI services never bypass it to access external systems or the database directly.
- Authentication and authorization information is shared through **JWT tokens** included in HTTP headers.
- External APIs are wrapped in dedicated integration modules in the backend to allow easy replacement (e.g., switching payment provider) without changing client code.
- Any future extension (e.g., additional web clients) must comply with the same REST/JSON and security standards to ensure interoperability.

3.4 Communications Interfaces

The VidAI system relies heavily on secure, real-time communication between the **mobile app**, **web application**, **Node.js backend**, **FastAPI AI microservices**, and **third-party APIs**. This section describes the communication protocols, message formats, security measures, and network requirements essential for system operation.

A. Network Communication Requirements

1. Primary Communication Protocol

- All communication (mobile, web, backend, AI microservices) must use **HTTPS** over **TLS 1.2+**.
- No unencrypted HTTP communication is permitted.
- Backend and AI microservices may use **HTTPS internal endpoints** or internal VPC routing.

2. Underlying Transport Protocol

- All communications use **TCP/IP** for reliable transmission.
- When required, internal services may use HTTP/2 for multiplexing.

3. Minimum Network Requirements

- Stable **3G/4G/5G or Wi-Fi**
- Minimum speed: **1 Mbps downstream** for normal operation
- Latency tolerance:
    - AI responses: **< 300 ms (ideal)**, **2–5 seconds max**
    - Standard API calls: **< 120 ms**

B. Message Formatting Standards

1. Data Format

- All API requests and responses use **JSON**.
- Images (vendor galleries, invitations, uploads) use **JPEG, PNG**, or **base64**.
- Optional AI image generation may return **base64 blobs** or **remote URLs**.

2. Character Encoding

- All textual communication uses **UTF-8 encoding**.

3. Booking & Payment Message Format

Includes fields such as:

- User ID
- Vendor ID
- Service type

- Booking date & time
- Payment token
- Transaction ID
- Status codes

4. AI Service Message Format

Incoming:

- User preferences
- Budget inputs
- Interaction history
- Chat messages

Outgoing:

- Recommendation arrays
- Budget breakdowns
- Chatbot NLP responses
- Smart planning suggestions

C. Server-to-Server Communication

1. Backend (Node.js) → AI Microservices (FastAPI)

- JSON-based REST API
- Internal HTTPS endpoints
- Response time target: **2–5 seconds**
- Retry attempts on failure

2. Backend → Database (MongoDB Atlas)

- Secure TLS-encrypted MongoDB protocol
- Access through **Mongoose** (Node) and optionally **PyMongo** (Python)
- No direct client access

3. Backend → Third-Party APIs

- Payment Gateways → Secure HTTPS REST
- SMS/Email Services → HTTP REST + SMTP over TLS
- Google Maps → HTTPS GET/POST
- Cloud Storage (S3/Cloudinary) → HTTPS PUT/GET
- AI Image APIs (optional) → HTTPS with JSON/image payloads

D. Communication with Notification Services

1. Push Notifications

- Delivered through **Firebase Cloud Messaging (FCM)**
- Requires device token (mobile only)
- JSON payloads for title, body, metadata

2. Email Communications

- Sent via **SMTP over TLS**
- Provider: SendGrid / Mailgun
- Supports HTML and plain-text templates

3. SMS Communications

- Sent via REST APIs to SMS gateway providers
- Used for OTPs, critical alerts, booking updates

E. Communication Security Requirements

1. Encryption

- TLS 1.2+ for all communications
- Sensitive fields (passwords, tokens) must be **hashed or encrypted**
- Payment data secured using gateway standards

2. Authentication Tokens

- JWT required for all API requests
- Tokens validated on every request
- Refresh tokens handled securely by backend

3. Secure Client Storage

- Mobile: Secure Enclave / Keychain / Encrypted Storage
- Web: HttpOnly cookies or secure local/session storage
- Logout clears all cached sensitive data

4. Security Protections

- Protection against replay attacks
- MITM prevention via enforced HTTPS
- Rate limiting on critical endpoints
- CORS policy enforced for **web application**
- CSRF protection for vendor/admin portal

F. Synchronization & Data Consistency

1. Real-Time Sync

- Bookings, notifications, and AI responses must sync instantly after confirmation
- Backend is the single source of truth

2. Client-Side Sync

- Local caching for quick responsiveness
- Auto-refresh on reconnection

3. Conflict Handling

- Latest timestamp wins (LWW model)
- Admin overrides supersede all client actions

4. Optional Real-Time Communication

- WebSockets or Socket.io can be used for:
    - Booking status updates
    - Live admin/vendor notifications
    - Chatbot streaming responses

(If not implemented initially, this remains future-scope.)

G. Browser-Based Communication (Vendor & Admin Web App)

1. Supported Browsers

- Chrome, Edge, Firefox, Safari (latest versions)

2. Communication Protocol

- **HTTPS**
- REST API with JSON payloads
- **CORS policy** defined on backend (allow only VidAI web domain)

3. Web Upload Interface

- HTML5 file upload APIs for images, documents
- Chunked uploads for large files (optional)

4. System Features

This section organizes the functional requirements of VidAI into major system features. Each feature below includes a **Use Case Description** summarizing actors, triggers, preconditions, normal flow, alternative flows, and postconditions. Detailed functional requirements (FRs) will follow in Section 5.

4.1 User Registration & Authentication

Use Case: UC-01 User Registration

Use Case Attribute	Description
Use Case ID	UC-01
Use Case Name	User Registration
Actors	End User, Vendor, Admin
Trigger	User selects “Sign Up.”
Preconditions	• Internet connection available • User does not already have an account
Postconditions	• New account created • JWT token issued
Main Flow	1. User opens sign-up form. 2. Enters required information. 3. Submits form. 4. Backend validates information. 5. System checks if account exists. 6. System creates account. 7. Returns success response + token.
Alternate Flow	A1: Invalid details → Show error. A2: Account already exists → Show message.
Exceptions	• DB write error • Backend unreachable

4.2 User Login

Use Case: UC-02 User Login

Use Case Attribute	Description
Use Case ID	UC-02
Use Case Name	User Login
Actors	End User, Vendor, Admin
Trigger	User selects “Login.”
Preconditions	Account exists
Postconditions	JWT token issued & session started
Main Flow	1. User enters credentials. 2. Backend verifies. 3. System returns JWT token. 4. User redirected to dashboard.
Alternate Flow	Invalid password → Error message.
Exceptions	Server down

4.3 Vendor Search & Filtering

Use Case: UC-03 Search Vendors

Use Case Attribute	Description
Use Case ID	UC-03
Use Case Name	Search Vendors
Actors	End User
Trigger	User opens vendor search page
Preconditions	User logged in
Postconditions	Vendors displayed
Main Flow	1. User enters query or filters. 2. Backend fetches matching vendors. 3. Results displayed.
Alternate Flow	No vendors found.
Exceptions	API timeout

4.4 View Vendor Details

Use Case: UC-04 View Vendor Profile

Use Case Attribute	Description
Use Case ID	UC-04
Use Case Name	View Vendor Details
Actors	End User
Trigger	User taps vendor card
Preconditions	Vendor exists
Postconditions	Vendor profile shown
Main Flow	1. User selects vendor. 2. System loads vendor details. 3. Shows gallery, pricing, reviews.
Alternate Flow	None
Exceptions	Vendor deleted or unavailable

4.5 Create Booking Request

Use Case: UC-05 Create Booking

Use Case Attribute	Description

Use Case ID	UC-05
Use Case Name	Create Booking Request
Actors	End User
Trigger	User selects “Book Now.”
Preconditions	User logged in
Postconditions	Booking created with Pending status
Main Flow	1. User selects date/time. 2. Adds notes. 3. System checks vendor availability. 4. Creates booking. 5. Vendor gets notification.
Alternate Flow	Vendor unavailable → Show error.
Exceptions	DB write error

4.6 Vendor Responds to Booking

Use Case: UC-06 Booking Response

Use Case Attribute	Description
Use Case ID	UC-06
Use Case Name	Vendor Responds to Booking
Actors	Vendor
Trigger	Vendor receives booking
Preconditions	Vendor logged in
Postconditions	Booking Approved or Rejected
Main Flow	1. Vendor opens request. 2. Reviews details. 3. Accepts or rejects. 4. User notified.
Alternate Flow	Vendor delays response
Exceptions	Network issues

4.7 Budget Planning

Use Case: UC-07 Budget Planner

Use Case Attribute	Description
Use Case ID	UC-07

Use Case Name	Budget Planning
Actors	End User
Trigger	User enters total budget
Preconditions	None
Postconditions	AI-generated budget breakdown
Main Flow	1. User enters budget. 2. System sends data to AI. 3. AI generates allocation. 4. System displays plan.
Alternate Flow	Invalid input
Exceptions	AI microservice down

4.8 AI Recommendations

Use Case: UC-08 AI Vendor Recommendations

Use Case Attribute	Description
Use Case ID	UC-08
Use Case Name	AI Recommendations
Actors	End User
Trigger	User opens “Recommended Vendors”
Preconditions	User preferences exist
Postconditions	Ranked vendor list shown
Main Flow	1. System sends preferences to AI. 2. AI model processes. 3. Returns ranked list.
Alternate Flow	No matching vendors
Exceptions	AI timeout

4.9 Digital Invitation Generator

Use Case: UC-09 Generate Invitation

Use Case Attribute	Description
Use Case ID	UC-09
Use Case Name	Digital Invitation Generator
Actors	End User
Trigger	User selects template

Preconditions	Template exists
Postconditions	Invitation exported
Main Flow	4. Select template. 5. Edit fields. 6. Preview. 7. Export/download.
Alternate Flow	Invalid inputs
Exceptions	Image processing failure

4.10 AI Chatbot

Use Case: UC-10 AI Chatbot Interaction

Use Case Attribute	Description
Use Case ID	UC-10
Use Case Name	AI Chatbot
Actors	End User
Trigger	User opens chat window
Preconditions	AI service must be online
Postconditions	AI response delivered
Main Flow	1. User enters message. 2. Backend forwards to AI NLP. 3. AI generates response. 4. User sees reply.
Alternate Flow	AI misunderstands query
Exceptions	AI unavailable

4.11 Vendor Service Management

Use Case: UC-11 Manage Vendor Services

Use Case Attribute	Description
Use Case ID	UC-11
Use Case Name	Vendor Service Management
Actors	Vendor
Trigger	Vendor logs into dashboard
Preconditions	Vendor verified
Postconditions	Services updated in system

Main Flow	5. Vendor edits pricing/pictures. 6. System validates. 7. Saves changes.
Alternate Flow	Missing fields
Exceptions	Upload error

4.12 Admin Vendor Verification

Use Case: UC-12 Vendor Verification

Use Case Attribute	Description
Use Case ID	UC-12
Use Case Name	Admin Vendor Verification
Actors	Admin
Trigger	Vendor submits documents
Preconditions	Vendor profile exists
Postconditions	Vendor approved/declined
Main Flow	1. Admin reviews docs. 2. Approves or rejects. 3. Vendor notified.
Alternate Flow	Insufficient documents
Exceptions	DB failure

4.13 Notification Delivery

Use Case: UC-13 Notifications

Use Case Attribute	Description
Use Case ID	UC-13
Use Case Name	Send Notifications
Actors	System
Trigger	Booking update / AI response / Admin action
Preconditions	Valid device token or contact
Postconditions	Notification delivered
Main Flow	1. Event triggers notification. 2. System sends SMS/Email/Push. 3. User receives message.
Alternate Flow	Delivery delays

Exceptions	Gateway failure

4.14 Payment Processing

Use Case: UC-14 Payment Processing

Use Case Attribute	Description
Use Case ID	UC-14
Use Case Name	Payment Processing
Actors	End User, Vendor, Payment Gateway
Trigger	User pays for service
Preconditions	Valid gateway configured
Postconditions	Transaction completed and status saved
Main Flow	1. User enters payment details. 2. Backend sends transaction request. 3. Gateway returns success/failure. 4. System updates booking.
Alternate Flow	Payment declined
Exceptions	Timeout or API failure

5. Nonfunctional Requirements

This section specifies the nonfunctional requirements essential for VidAI's performance, safety, security, and overall quality. These requirements define how the system must behave and the constraints under which it must operate. All requirements are written to be measurable, verifiable, and aligned with customer expectations and development constraints.

5.1 Performance Requirements

1. **Response Time**
   - The system shall provide responses to standard user actions (browsing vendors, loading profiles, performing searches) within **2 seconds** under normal network conditions.
   - AI-related actions (budget planning, recommendation generation, chatbot responses) shall be completed within **2–5 seconds**.
2. **Concurrent User Support**
   - The system shall support at least **500 concurrent users** during peak load for the FYP deployment environment.
3. **Booking Operations Throughput**

- Vendor booking requests shall be processed and recorded within **3 seconds** after submission.

4. **Mobile Application Performance**
    - The app shall load and become interactive within **3 seconds** on mid-range Android devices (≥ 2GB RAM).
5. **Search Performance**
    - Search results shall return a minimum of **10 vendor listings per second**.
6. **Data Sync Frequency**
    - The system shall update booking status and notifications in real time, with a synchronization delay of **no more than 1 second**.

5.2 Safety Requirements

1. **Data Protection Against Loss**
    - Daily automated database backups shall be maintained via MongoDB Atlas to prevent data loss.
2. **Critical Failure Handling**
    - In case of backend failure, the system shall provide users with error messages and retry options instead of crashing.
    - No irreversible actions (such as deleting vendor accounts or bookings) shall be allowed without confirmation dialogs.
3. **AI Response Safety**
    - The AI chatbot shall not provide harmful, misleading, or unsafe advice (e.g., financial scams, spam, discriminatory messages).
4. **Secure Payments**
    - Payment-related operations must rely on certified third-party gateways (e.g., JazzCash/EasyPaisa/Stripe) to prevent risks involving sensitive financial data.
5. **Vendor Verification Safety**
    - Only verified vendors (submitted CNIC or business documentation) may appear publicly to ensure user safety and prevent fraud.
6. **External Regulation Compliance**
    - The system shall comply with local Pakistani e-commerce safety expectations and payment provider regulations.

5.3 Security Requirements

1. **Authentication & Authorization**
    - All users shall authenticate using secure JWT tokens.

    - Role-based access control (RBAC) must ensure users cannot access vendor/admin functions.
2. **Encryption Requirements**
    - All data transmitted between client and server shall use **TLS 1.2+**.
    - Passwords must be hashed using **bcrypt** or **Argon2**.
3. **Data Privacy**
    - Personal data (user contact info, vendor documents, booking details) shall not be shared without user consent.
    - Sensitive data stored on the device shall be protected using OS-level secure storage.
4. **API Security**
    - All API routes must validate inputs to prevent SQL/NoSQL injection.
    - Rate-limiting shall be used to prevent brute-force attacks.
5. **Access Logging**
    - The system shall log all login attempts, vendor changes, admin actions, and booking modifications for auditing.
6. **Session Security**
    - JWT tokens shall expire automatically after a defined period (e.g., 24 hours).
    - Refresh tokens shall be securely stored and validated.

5.4 Software Quality Attributes

5.4.1 Usability

- The system shall be intuitive enough for a first-time user to complete basic tasks (search vendor → open profile → send booking request) within **5 minutes**.
- UI elements shall follow Material Design or equivalent modern mobile UX standards.

5.4.2 Reliability

- The system shall provide **99% uptime** under normal hosting conditions.
- Faulty network operations shall trigger retry mechanisms.

5.4.3 Maintainability

- Code must follow modular architecture principles to ensure developers can add new vendor categories without restructuring the system.
- All API endpoints shall be documented using **Swagger/OpenAPI**.

5.4.4 Portability

- The mobile application must run on both Android 8.0+ and iOS 13+.

- Backend must be deployable on any cloud platform supporting Node.js or Python.

5.4.5 Interoperability

- The system shall communicate with at least four external services:
    1. Payment gateway
    2. Google Maps API
    3. SMS/Email services
    4. Cloud storage/image APIs
- All interoperability must use standardized RESTful communication with JSON.

5.4.6 Adaptability

- The system shall allow adding new event types (e.g., engagement, mehndi) without functional redesign.
- AI models shall support retraining with new data.

5.4.7 Robustness

- The system must not crash on unexpected user input or network loss.
- All forms shall include field validation to prevent incorrect submissions.

5.4.8 Testability

- The backend shall support automated unit tests for at least **60% of major modules**.
- Components shall expose predictable behaviors for automated testing tools.

5.5 Business Rules

1. **Vendor Verification Rule**
    - Only verified vendors may appear in public search results.
2. **Booking Conflict Rule**
    - A vendor cannot accept two bookings for overlapping time slots.
3. **User Eligibility Rule**
    - Only registered users may initiate bookings or generate invitations.
4. **Price Accuracy Rule**
    - Vendors must update accurate service pricing; misleading pricing may result in admin suspension.
5. **Refund Policy Rule**
    - Refunds or cancellations must follow the policies of the integrated payment gateway.
6. **Admin Override Rule**

    - Admins may deactivate users or vendors who violate policies.

7. **AI Transparency Rule**
    - AI recommendations must be explainable (showing why a vendor was recommended).

6. Other Requirements

This section defines additional system requirements that do not fall under previous categories but are essential for proper functionality, compliance, deployment, and long-term scalability of VidAI.

6.1 Database Requirements

1. **Database Type**
    - The system shall use **MongoDB Atlas**, a cloud-based NoSQL document database with full support for clustering, backups, and security.
2. **Data Consistency Requirements**
    - Read-heavy operations (vendor search, browsing, recommendations) may use **eventual consistency**.
    - Booking creation and payment-related updates must use **strong consistency** to prevent double booking.
3. **Backup & Recovery**
    - Automated daily backups must be enabled and retained for **at least 7 days**.
    - Recovery Time Objective (RTO) shall not exceed **30 minutes** for FYP-scale data.
4. **Scalability**
    - MongoDB must support **horizontal scaling via sharding** as the number of vendors and users grows.
    - Collections must support indexing for optimized search performance.
5. **Schema Requirements**
    - Collections such as **User**, **Vendor**, **Booking**, **Invitation**, and **AI Preference** must follow standardized schema rules.
    - Schema definitions will be formally included in the **SDS (Data Model section)**.

6.2 Internationalization & Localization Requirements

1. **Language Support**
    - Default UI language is **English**.
    - The system design must support future Urdu localization, including:
        - Right-to-left (RTL) text layout
        - Expanded UI spacing for Urdu script
2. **Date & Time Formatting**

- The system shall use **Pakistan Standard Time (PST)** as the default timezone.

3. **Currency Format**
   - All financial values must be displayed in **PKR (Rs)**.
4. **Character Encoding**
   - UTF-8 must be used throughout the system for English + Urdu compatibility.

6.3 Legal & Regulatory Requirements

1. **Data Protection Compliance**
   - VidAI must follow general digital privacy principles, including:
     - Collecting minimal user data
     - Securing data at rest and in transit
     - Allowing users to request account deletion
2. **Consumer Protection**
   - Vendor service descriptions must be accurate and not misleading.
   - Admin must moderate fake or fraudulent vendor accounts.
3. **Payment Gateway Regulations**
   - All payment transactions must follow rules defined by JazzCash, EasyPaisa, Stripe, or relevant providers.
   - VidAI must not store sensitive payment card information.
4. **Intellectual Property Rights**
   - All digital invitation templates and AI-generated images must not violate copyright laws.
   - Proper attribution must be followed for any external template sources.
5. **Third-Party API Usage Policies**
   - VidAI must comply with usage limits, rate limits, and policies of services such as Google Maps, Firebase, Cloudinary, and SMS/email providers.

6.4 Reuse Objectives

1. **Component Reusability**
   - UI components (buttons, cards, navigation bars, modals) must be implemented as reusable components.
   - Shared utilities (validators, form handlers, API wrappers) must be reusable across mobile and web platforms.
2. **Backend & AI Reusability**
   - Authentication, vendor management, booking management, and notification modules must be structured for reuse in future projects.

- AI microservices (recommendation engine, chatbot, budget planner) shall be modular and reusable across different domains.

3. **API Reuse**
   - REST APIs must follow consistent naming conventions and CRUD patterns for reuse across both mobile and web applications.

6.5 Logging & Monitoring Requirements

1. **Backend Logging**
   - All API requests, errors, warnings, and admin actions shall be logged with timestamps, user IDs, and status codes.
2. **AI Service Logging**
   - AI microservices must log:
     - Incoming requests
     - Processing duration
     - Errors and fallback behavior
     - Model-related warnings (e.g., missing data)
3. **Monitoring Tools**
   - Cloud-based monitoring tools (AWS CloudWatch, GCP Monitoring, or similar) must track:
     - Server uptime
     - AI microservice health
     - Database performance
     - API error rates
     - Storage usage
4. **Log Retention**
   - Logs must be retained for at least **30 days** for debugging and analysis.

6.6 Deployment Requirements

1. **Hosting Environment**
   - Backend (Node.js) and AI microservices (FastAPI) must be deployable using:
     - Docker containers
     - Cloud-managed services (e.g., AWS ECS, Railway, Render)
2. **Web Application Deployment**
   - Vendor/Admin web application (React.js) must be deployable via:
     - Cloud hosting
     - Static site hosting (e.g., Netlify, Vercel)
3. **Mobile Application Distribution**
   - VidAI must support deployment via:

    - Android APK/AAB for testing
    - Optional iOS deployment via TestFlight (if hardware available)

4. **CI/CD Requirements (Optional for FYP)**
    - Automated deployment pipelines may be used to reduce manual errors.

6.7 Future Enhancement Requirements (Non-Binding)

(Not required for FYP but included for long-term product development.)

1. Full Urdu & Multilingual UI
2. Vendor Analytics Dashboard
3. In-app Chat with Vendors
4. Fully Automated Wedding Timeline Generator
5. Blockchain/Smart Contract-based Payment System
6. Integration with bridal fashion catalogs
7. User-to-vendor video consultation scheduling

Software Design Specification

1. Introduction

1.1 Purpose

The purpose of this Software Design Specification (SDS) is to present a complete and detailed technical description of the internal design, architecture, components, data structures, interfaces, and operational behavior of the **VidAI Wedding Planning System**. While the SRS defines *what* the system must do, this SDS explains *how* the system will be engineered to meet those requirements.

This SDS reflects the updated system architecture, where:

- Mobile App (React Native) serves end users (bride/groom/family).
- Web Application (React.js) serves vendors and administrators.
- Backend API (Node.js/Express) acts as the unified server for both applications.
- AI Microservices (FastAPI / Python) operate as independent modules for recommendations, planning, and chatbot responses.
- MongoDB Atlas functions as the shared cloud database for all subsystems.

Scope of the SDS

This document includes detailed design specifications for:

- Overall system architecture (high-level & low-level)
- Component/module decomposition for mobile, web, backend, and AI services
- API design and inter-service communication
- Data Flow Diagrams (DFD Level 0, Level 1, Level 2)
- Class diagrams and object models
- Database schema and Entity-Relationship Diagram (ERD)
- Sequence diagrams for all major workflows
- Activity diagrams for user-facing processes
- Security architecture and constraints
- Deployment architecture for cloud hosting
- Technology stack and development environment specifications

Systems Covered in This Design Document

This SDS describes the detailed technical design of all VidAI subsystems:

- React Native Mobile App (User-facing application)
- React.js Web App (Vendor & Admin portal)

- Node.js Backend API Server (Unified REST API)
- FastAPI-based AI Microservices (Recommendation engine, budget planner, chatbot)
- MongoDB Atlas Cloud Database
- Third-party integrations (payment gateways, notifications, storage, maps)

Intended Audience

This document is intended for:

- **Developers** implementing system components
- **Project managers** monitoring progress
- **QA testers** writing test cases
- **Supervisors and evaluation committees** reviewing technical correctness
- **Future maintainers** extending or scaling the system

1.2 Document Conventions

The following standards, notations, and formatting conventions are used throughout this Software Design Specification (SDS) to maintain clarity, uniformity, and technical correctness across all architectural and design artifacts.

A. Notation Standards

1. UML Notation

- All structural and behavioral diagrams follow **UML 2.x standards**, including:
    - Class diagrams
    - Sequence diagrams
    - Activity diagrams
    - Component diagrams
    - Deployment diagrams
- Diagrams are created using modeling tools such as **Mermaid.js**, **PlantUML**, **Draw.io**, or equivalent UML-compliant software.
- Architectural stereotypes such as <<frontend>>, <<backend>>, <<microservice>>, <<database>>, and <<API>> are used where appropriate.

2. DFD (Structured Analysis) Notation

- Data Flow Diagrams follow **Yourdon/DeMarco notation**.
- Context diagrams use standardized symbols for:
    - External entities
    - Processes
    - Data stores

    - Data flows
- Level-0 and Level-1 DFDs follow consistent hierarchical numbering.

3. ERD (Database) Notation

- Entity-Relationship Diagrams use **Crow's Foot Notation** to depict relationships (1-1, 1-N, N-N).
- MongoDB document structures are represented using:
    - Document models
    - Embedded subdocuments
    - Nested arrays
- Primary keys, foreign keys, and collection names follow consistent naming conventions.

B. Typographical Conventions

- **Bold text**

  Used for section headings, subsystem/component names, and architectural elements.
- *Italic text*

  Used for notes, assumptions, and explanatory remarks.
- Monospaced text

  Used for:
    - API endpoints (/api/vendors)
    - JSON structures
    - Code fragments
    - Data model attributes
    - Configuration examples

C. Requirement & Component Labeling

To ensure uniform referencing across diagrams and design sections:

- **Architectural Components:**

  AC-x (e.g., AC-03 Vendor Management Service)
- **Data Elements:**

  DE-x (e.g., DE-05 User Preference Vector)
- **Interfaces & APIs:**

  IF-x (e.g., IF-02 AI Microservice API)
- **Modules:**

MOD-x (e.g., MOD-01 Authentication Module)

- **Diagrams:**

  All UML/DFD/ERD diagrams use consistent names across SDS sections.

D. Priority and Inheritance Rules

- All design requirements follow the **MoSCoW priority model:**
  - **M** — Must Have
  - **S** — Should Have
  - **C** — Could Have
  - **W** — Will Not Have (in current release)
- Priorities are **not inherited** from high-level SRS requirements. Each SDS design element specifies its own priority level.

E. Formatting Standards

- **Page numbering** follows hierarchical format:

  Chapter.Section.Page
- **Figures** follow the naming format:

  Figure X.Y — Title of Diagram with a short explanatory caption.
- **Tables** follow consistent column headers and numbering.
- **Naming Conventions:**
  - API endpoints use REST conventions (lowercase, hyphenated names).
  - Database collections use lowercase plural names (e.g., vendors, users).
  - Classes and services use PascalCase (e.g., VendorService).
  - Variables and JSON fields use camelCase.

F. Units and Terminology

- **Currency:** All financial values use PKR (Rs).
- **Time Format:** 24-hour format, PST (UTC+5).
- **Terminology:**
  - **"AI Engine"** refers collectively to the Recommendation Engine, Budget Planner, and NLP Chatbot.
  - **"Frontend"** refers to both the React Native mobile app and React web app.
  - **"Backend API"** refers to the Node.js/Express service layer.
  - **"Microservice"** refers to FastAPI-based independent AI services.

2. Software Architecture

This section provides a complete overview of the software architecture of the **VidAI Wedding Planning System**, describing the architectural patterns, subsystem decomposition, module responsibilities, and communication flow between components. The architecture is designed for modularity, maintainability, scalability, and ease of future extension.

VidAI follows a **multi-tier, service-oriented, microservice-enhanced architecture** consisting of:

- A **mobile application** (React Native) for end users
- A **web application** (React.js) for vendors and administrators
- A **backend API service** (Node.js / Express)
- One or more **AI microservices** (FastAPI / Python)
- A **cloud-hosted NoSQL database** (MongoDB Atlas)
- Various **external services** for payments, notifications, maps, and media storage

2.1 Architectural Patterns and Styles

The architecture of VidAI incorporates multiple proven design patterns:

1. Client–Server Architecture

- The **React Native mobile app** and **React web portal** act as clients.
- The **Node.js backend** acts as the central server handling:
  - Business logic
  - Data access
  - Authentication
  - Communication with AI microservices & third-party APIs

2. Layered (N-Tier) Architecture

The backend is organized into logical layers:

- **Presentation Layer**

  Exposed as REST APIs consumed by mobile and web clients.

- **Business Logic Layer**

  Implements core workflows (booking, vendor management, notifications, AI orchestration).

- **Data Access Layer**

  Handles all interactions with MongoDB Atlas through schemas/models.

3. Microservice Architecture (AI Components)

- All AI-intensive features (recommendations, budget planning, chatbot NLP) reside in **independent FastAPI microservices**, each with its own REST endpoints.
- Microservices can scale independently and evolve separately from main backend logic.

4. RESTful API Pattern

- The system exposes **JSON-based REST APIs** for:
    - Mobile app
    - Vendor/admin portal
    - AI microservices

5. Modular Backend Architecture

Although deployed as a single backend service (for FYP feasibility), the backend is internally decomposed into modules:

- Authentication
- Vendor Management
- Booking Management
- Budgeting
- Notification Integration
- AI Integration
- Storage Integration

Internal modularity ensures maintainability and clear separation of responsibilities.

2.2 High-Level Architectural Overview

VidAI consists of the following major subsystems:

A. Mobile Application (React Native) — End User Client

- Offers a complete set of user-facing features:
    - Registration, login
    - Vendor browsing & filtering
    - Bookings
    - Budget management
    - Digital invitations
    - Chatbot
- Communicates exclusively with backend via secure HTTPS.

B. Web Application (React.js) — Vendor & Admin Portal

- **Vendor Portal** enables:

    - Managing services, packages, and pricing
    - Updating availability
    - Handling bookings
- **Admin Portal** enables:
    - Vendor verification
    - Platform monitoring
    - User management
    - Activity moderation

C. Backend API Service (Node.js / Express)

The backend acts as the **core orchestrator** of the system:

- Handles user/vendor/admin authentication (JWT)
- Enforces RBAC
- Processes booking logic
- Communicates with AI microservices
- Stores and retrieves data from MongoDB Atlas
- Interfaces with third-party services:
    - Payment gateways
    - Cloud Storage
    - Firebase notifications
    - Google Maps API

D. AI Microservices (FastAPI / Python)

Independent services responsible for:

- Vendor recommendations
- Personalized wedding plan suggestions
- Budget optimization
- Chatbot NLP responses

Each microservice exposes REST endpoints such as:

- /recommend/vendors
- /plan/budget
- /chat/ask

Backend communicates with them using internal secure API calls.

E. Database Layer (MongoDB Atlas)

Stores all persistent data:

- Users
- Vendors
- Bookings
- Reviews & Ratings
- Invitation templates
- AI logs & metadata

Provides replication, backups, scaling, and indexing.

F. External Service Integrations

Used for extending system capabilities:

- **Payment Gateways:** JazzCash, EasyPaisa, Stripe
- **Notifications:** FCM, SMS APIs, SendGrid
- **Maps & Location:** Google Maps Places API
- **Media Storage:** Cloudinary or AWS S3
- **Optional AI Image APIs:** Digital invitation generation

2.3 Architectural Elements / Modules / Components and Responsibilities

Below is the detailed breakdown of each subsystem.

2.3.1 Mobile Client (React Native)

Responsibilities:

- User Registration & Login
- Vendor search, filtering, browsing
- Vendor profiles with price, gallery, availability
- Booking creation and tracking
- Budget input & AI-based planning interface
- Digital invitation creation & preview
- Chatbot communication
- Push notification handling
- Local caching of session & temporary data

2.3.2 Vendor/Admin Web Portal (React.js)

Vendor Responsibilities:

- Manage profile, pricing, catalog, availability

- Upload images to portfolio
- Respond to bookings
- View analytics (optional future addition)

Admin Responsibilities:

- Verify vendor documents
- Approve/Reject vendor listings
- Monitor bookings & user activity
- Suspend/ban accounts
- View platform statistics

2.3.3 Authentication & User Management Module (Backend)

Responsibilities:

- Registration (User/Vendor/Admin)
- Secure password hashing
- JWT generation and refresh
- Role-based access control
- OTP & account recovery

2.3.4 Vendor Management Module (Backend)

Responsibilities:

- CRUD operations for vendor profiles
- Vendor service listings & pricing
- Vendor categories
- Search & filtering logic
- Vendor verification status

2.3.5 Booking Management Module (Backend)

Responsibilities:

- Booking creation and modification
- Conflict detection (overlapping time slots)
- Vendor acceptance/rejection workflows
- Status updates
- Payment integration
- Notification triggers

2.3.6 Budget & Expense Management Module (Backend)

Responsibilities:

- Store initial user budgets
- Track allocated vs. spent amounts
- Request optimized breakdowns from AI Budget Planner
- Provide charts/data for frontend visualizations

2.3.7 AI Recommendation & Planning Microservice

Responsibilities:

- Provide personalized vendor recommendations
- Analyze budgets & generate intelligent allocations
- Improve suggestions using behavioral data
- Return structured outputs to backend

2.3.8 Chatbot / NLP Microservice

Responsibilities:

- Interpret user messages
- Understand intent and entities
- Provide responses or launch actions (search vendors, give suggestions)
- Gracefully degrade to fallback messages

2.3.9 Invitation Design & Media Service

Responsibilities:

- Manage static or AI-generated invitation templates
- Store templates in Cloud Storage
- Generate shareable invitation image URLs
- Optional: call external AIGC template generator

2.3.10 Notification Service Integration

Responsibilities:

- Push notifications (FCM)
- SMS OTP & alerts
- Email confirmations
- Retry logic on failure

2.3.11 Admin & Moderation Module

Responsibilities:

- Vendor verification workflow

- User/vendor management
- Fraud detection flags
- Dashboard and reporting

2.3.12 Data Access & Persistence Layer (Backend)

Responsibilities:

- Define database schemas for Users, Vendors, Bookings, etc.
- Perform CRUD operations
- Enforce validation rules
- Optimize queries using indexes

2.3.13 Integration Layer (Backend)

Responsibilities:

- Payment Gateway
- Maps API
- Cloud Storage
- Notification APIs
- AI microservice interfaces

Provides clean functions such as:

processPayment(), uploadImage(), getLocation(), callAI()

2.4 Responsibilities Summary

Subsystem	Primary Responsibilities
Mobile App	UX, client logic, vendor browsing, bookings, budget tools
Vendor/Admin Web Portal	Vendor management, admin moderation
Backend API	Business logic, authentication, orchestration, data access
AI Microservices	Recommendations, budgeting, chatbot
Database	Persistent, scalable cloud storage
External Services	Payments, maps, notifications, media hosting

---

2.5 Architecture Diagrams

2.5.1 Logical Architecture (Component Diagram)

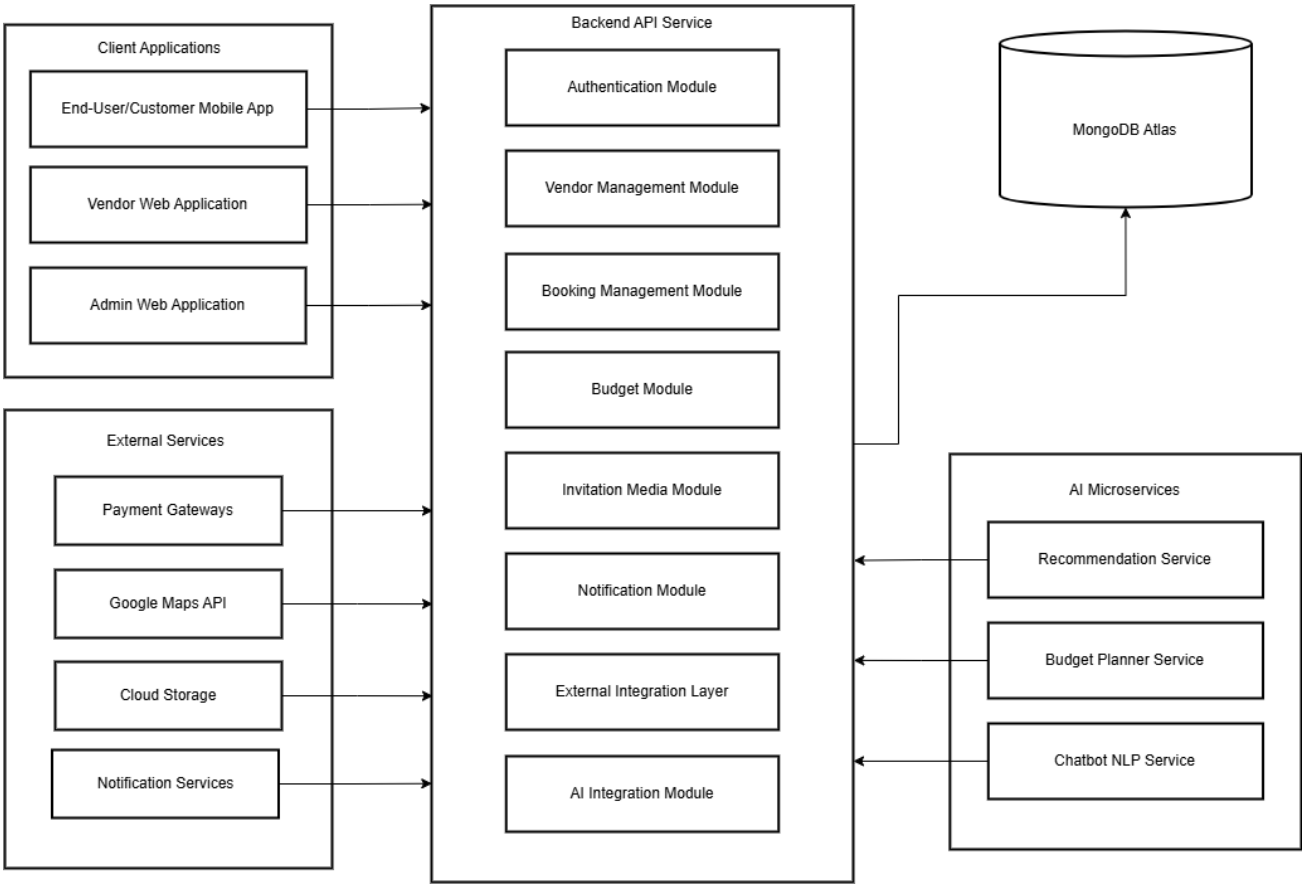


Figure 2.5.1 – Component Diagram

2.5.2 Physical Architecture (Deployment Diagram)

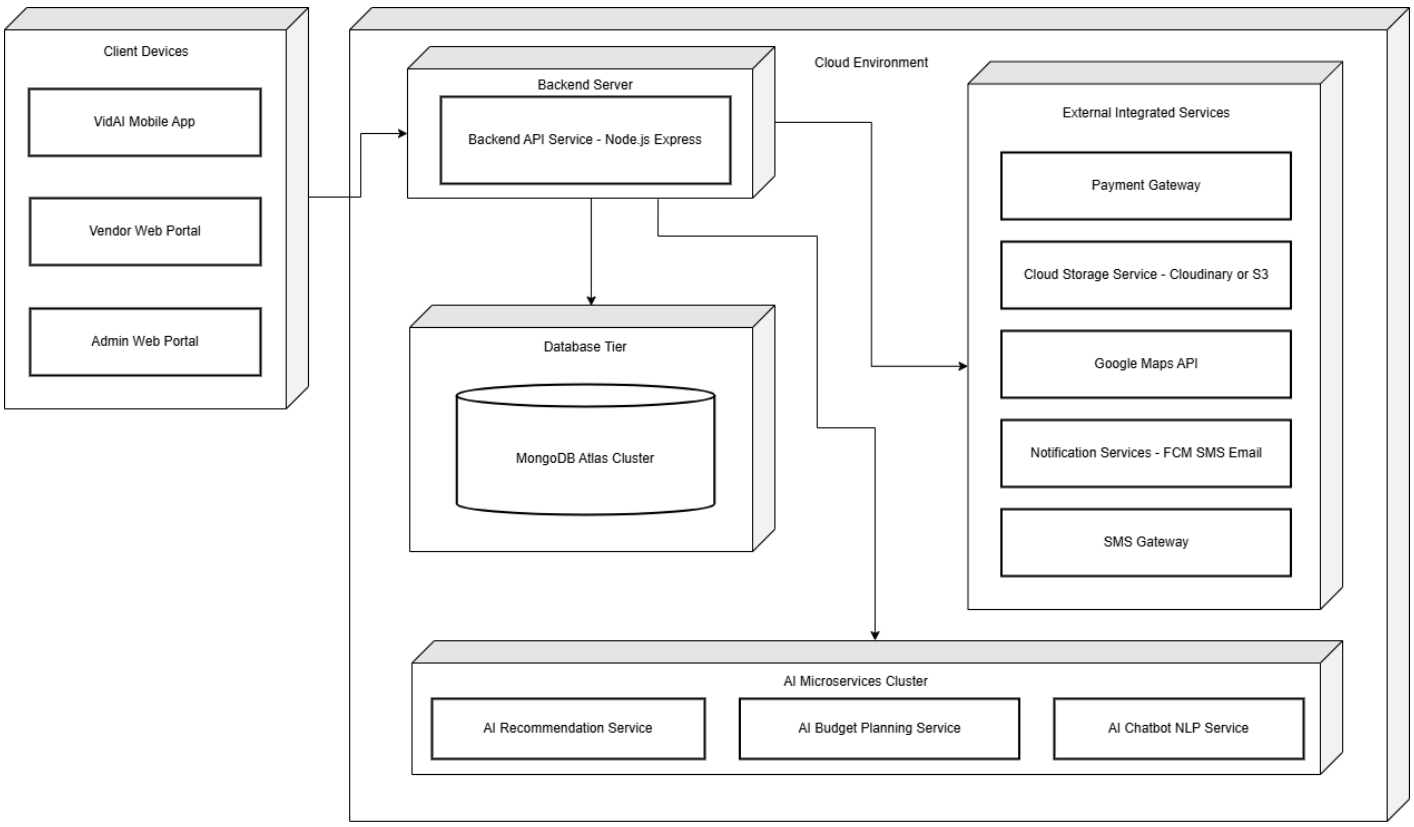


Figure 2.5.2 - Deployment Diagram

2.5.3 Description Summary

- Mobile layer: UI/UX, API communication
- Backend: business logic, validation, integration orchestration
- AI layer: independent FastAPI microservices
- Database: MongoDB Atlas cluster
- External services: payment, maps, notifications, storage

3. Detailed Design

This section presents the detailed design of the VidAI system using Object-Oriented Design (OOD). The design is described from multiple architectural views: logical, dynamic, deployment, development, data, and user interface. Together, these views provide a complete blueprint for implementation.

3.1 Logical View

3.1.1 UML Class Diagram

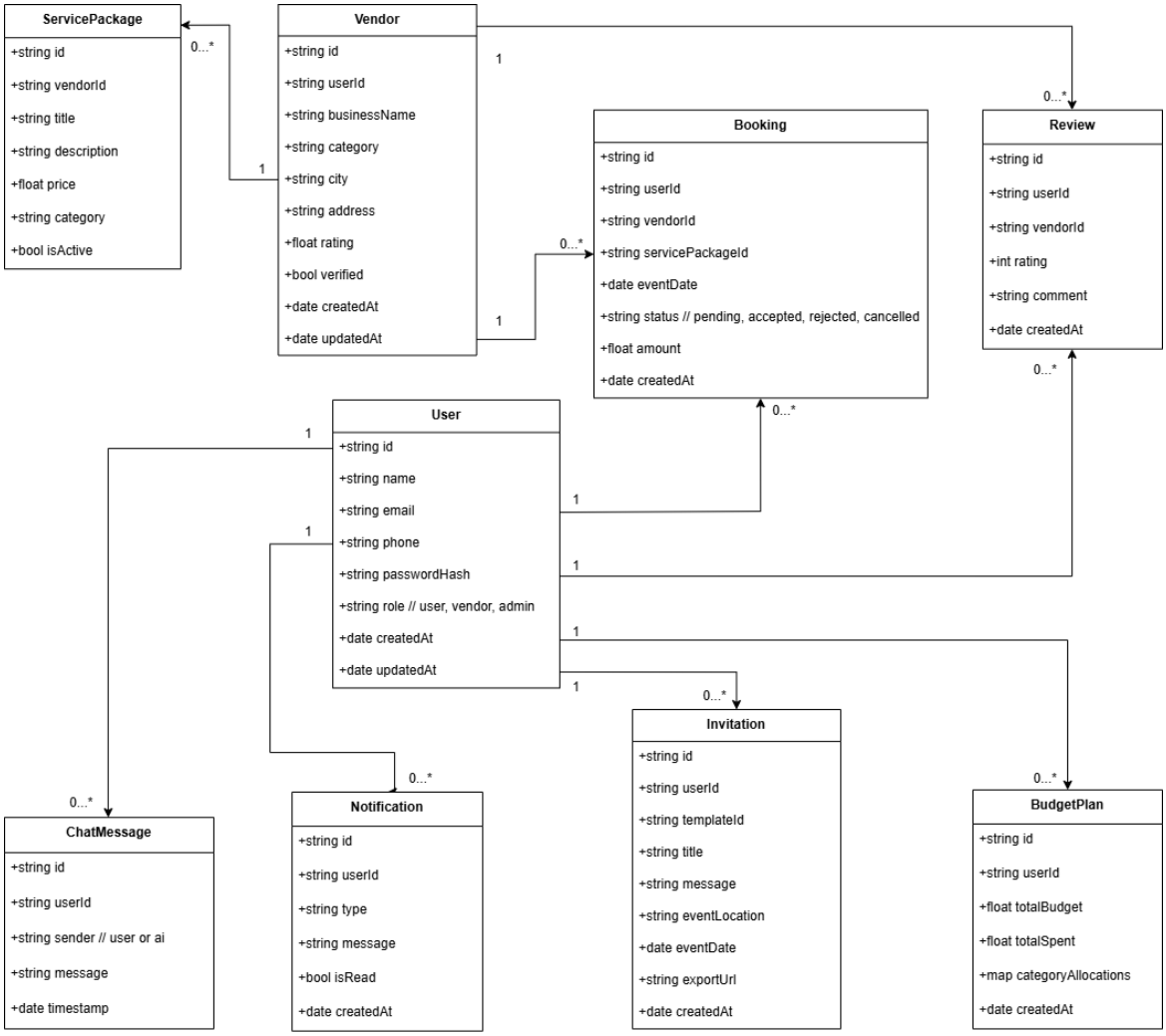


Figure 3.1.1 - Class Diagram

3.2 Dynamic View

The dynamic view describes **runtime behavior** of the system using sequence diagrams and (optionally) activity diagrams.

3.2.1 Sequence Diagrams

3.2.1.1 Sequence Diagram – Book Vendor

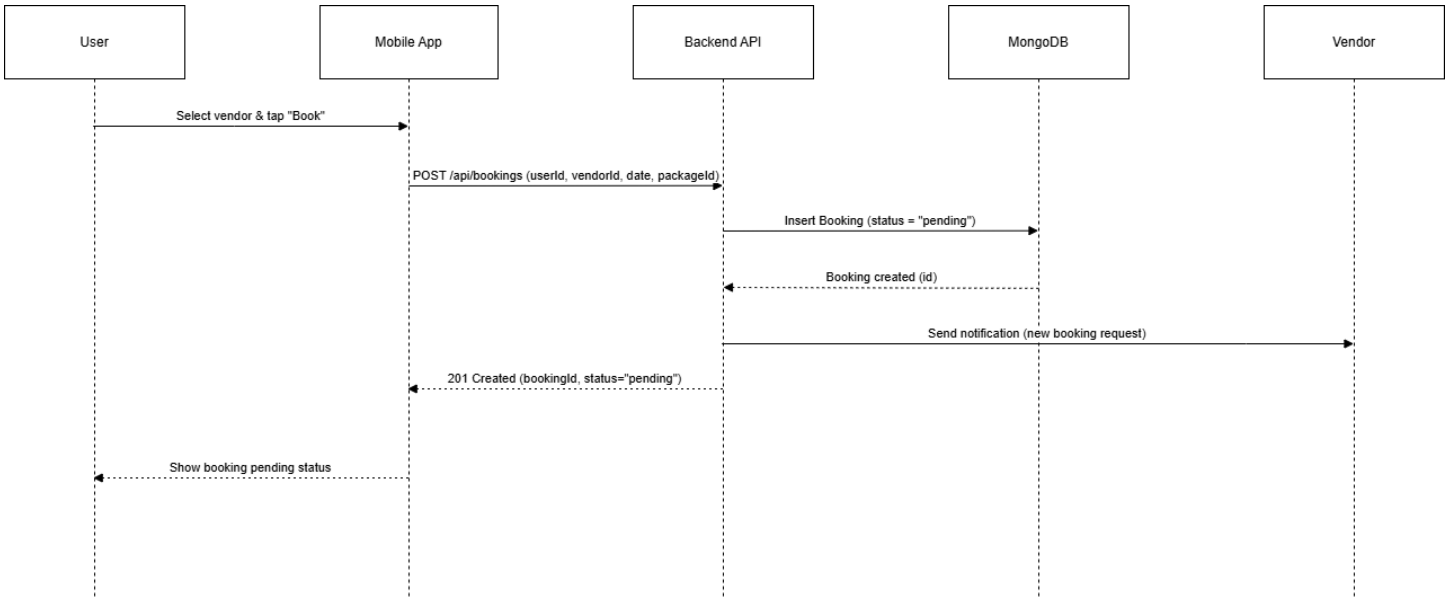


Figure 3.2.1.1 - Sequence Diagram – Book Vendor

3.2.1.2 Sequence Diagram – AI Recommendation Request

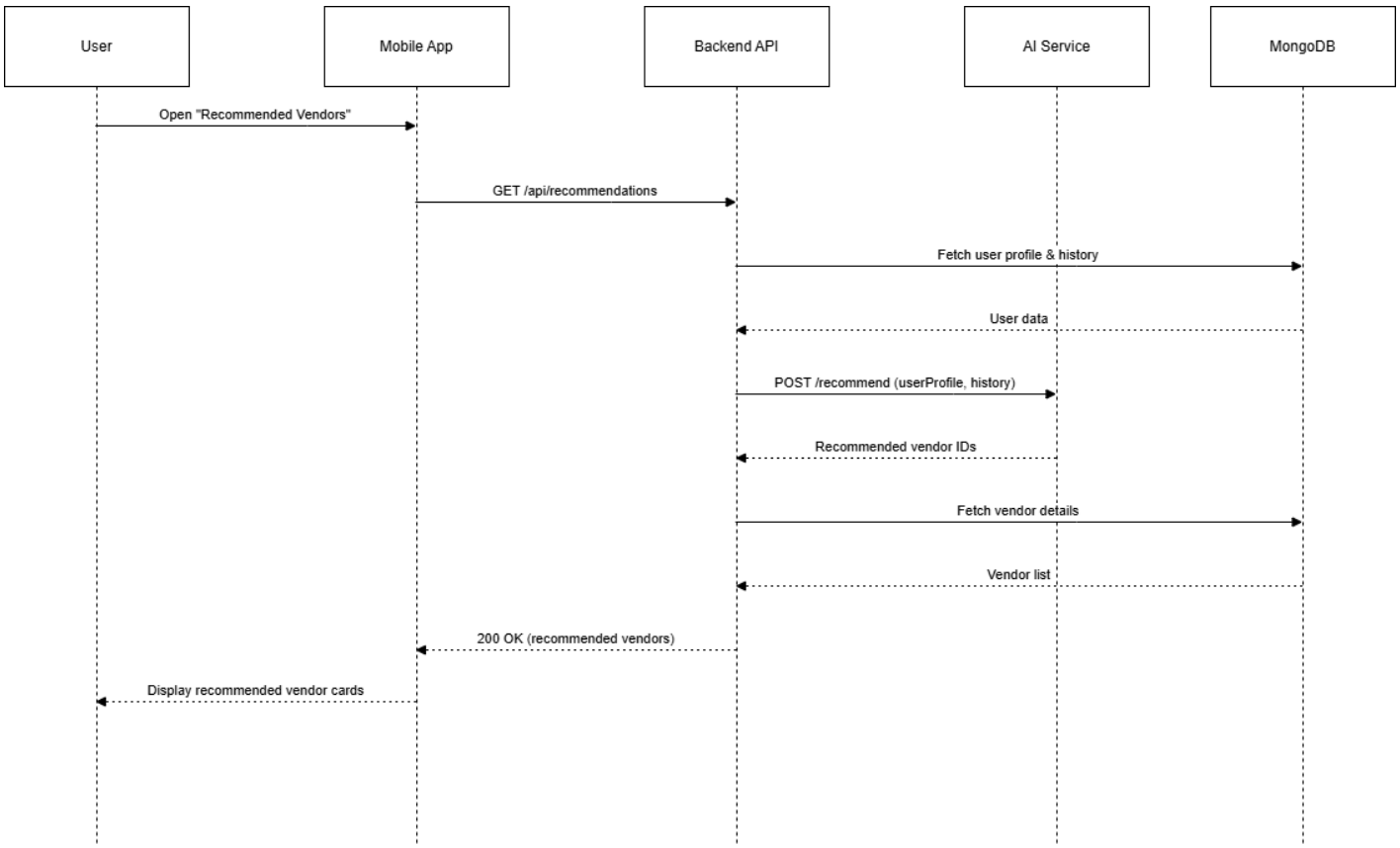


Figure 3.2.1.2 - Sequence Diagram – AI Recommendation Request

3.2.1.3 Sequence Diagram – User Login

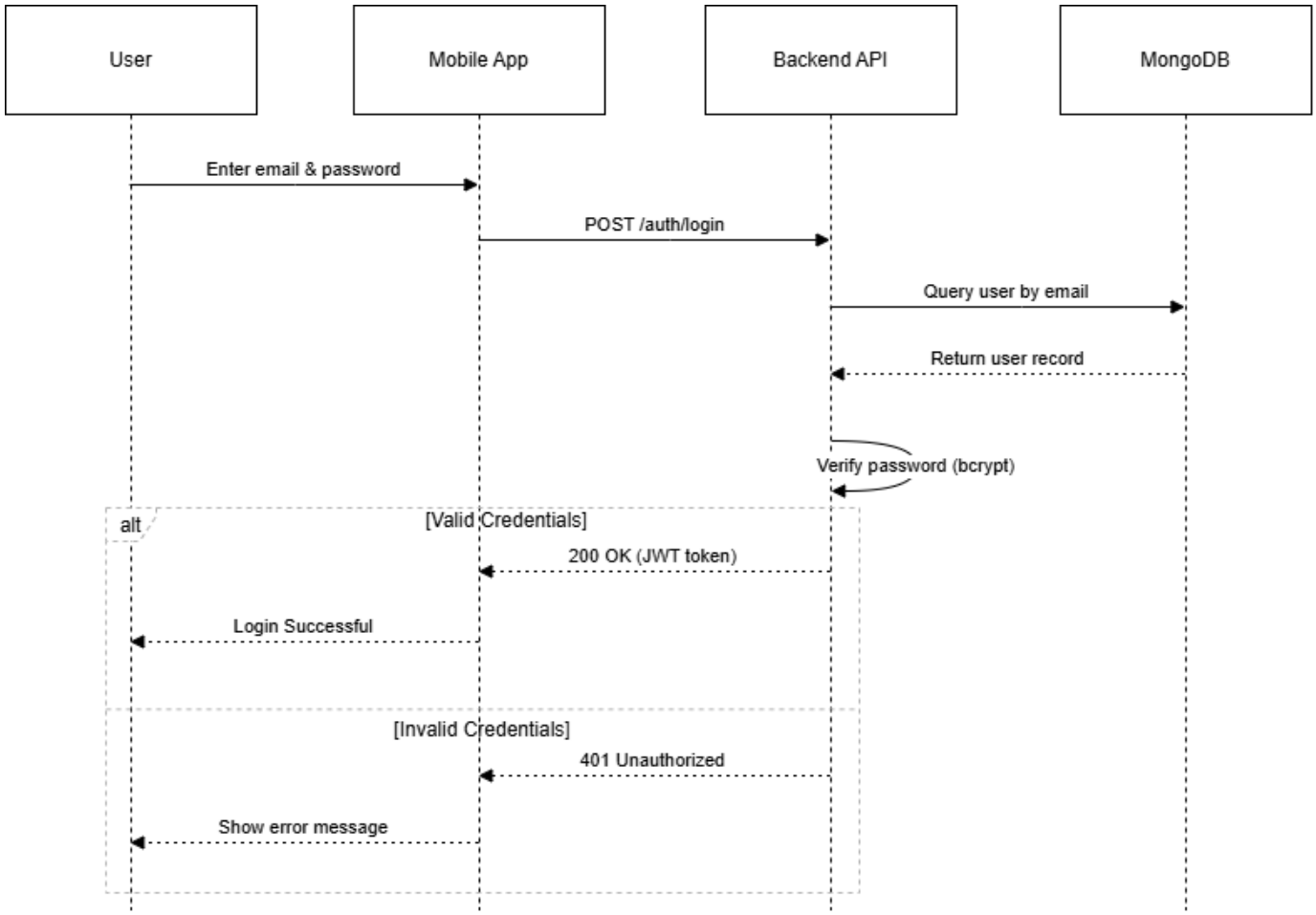


Figure 3.2.1.3 - Sequence Diagram – User Login

3.2. 1.4 Sequence Diagram – Vendor Accepts Booking

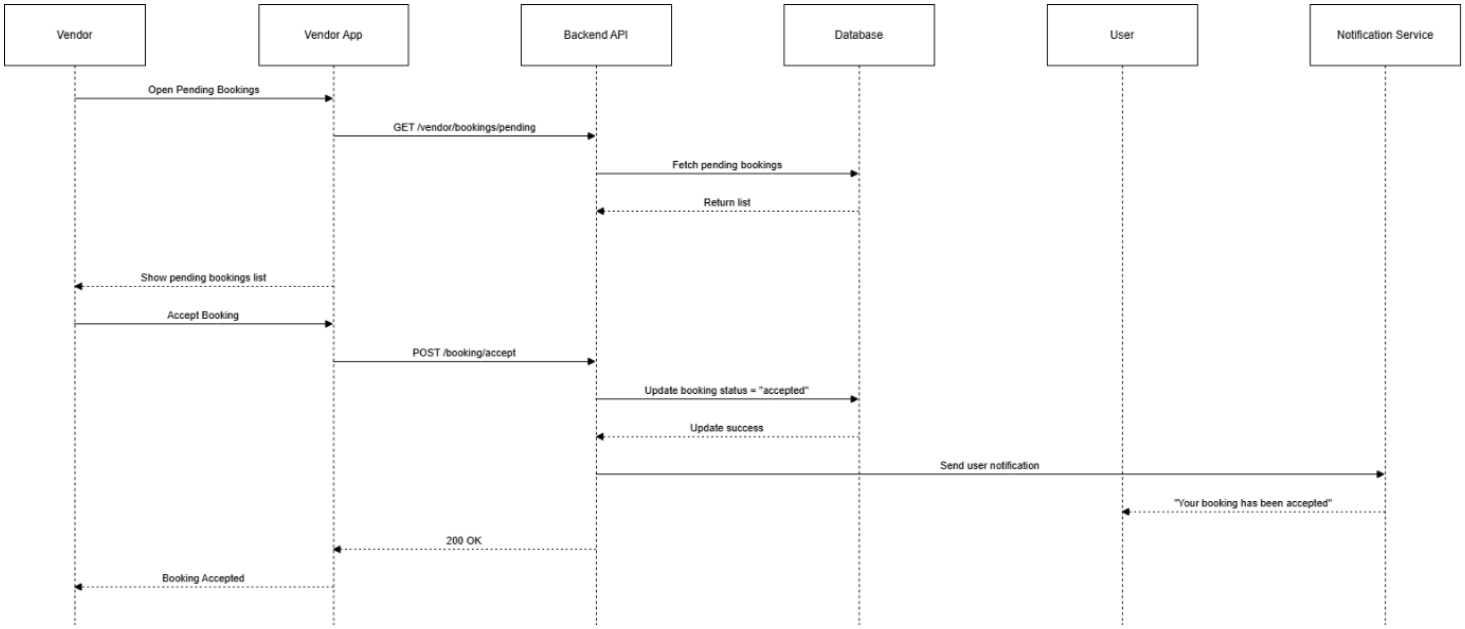


Figure 3.2.1.4 - Sequence Diagram – Vendor Accepts Booking

3.2.1.5 Sequence Diagram – Create Digital Invitation

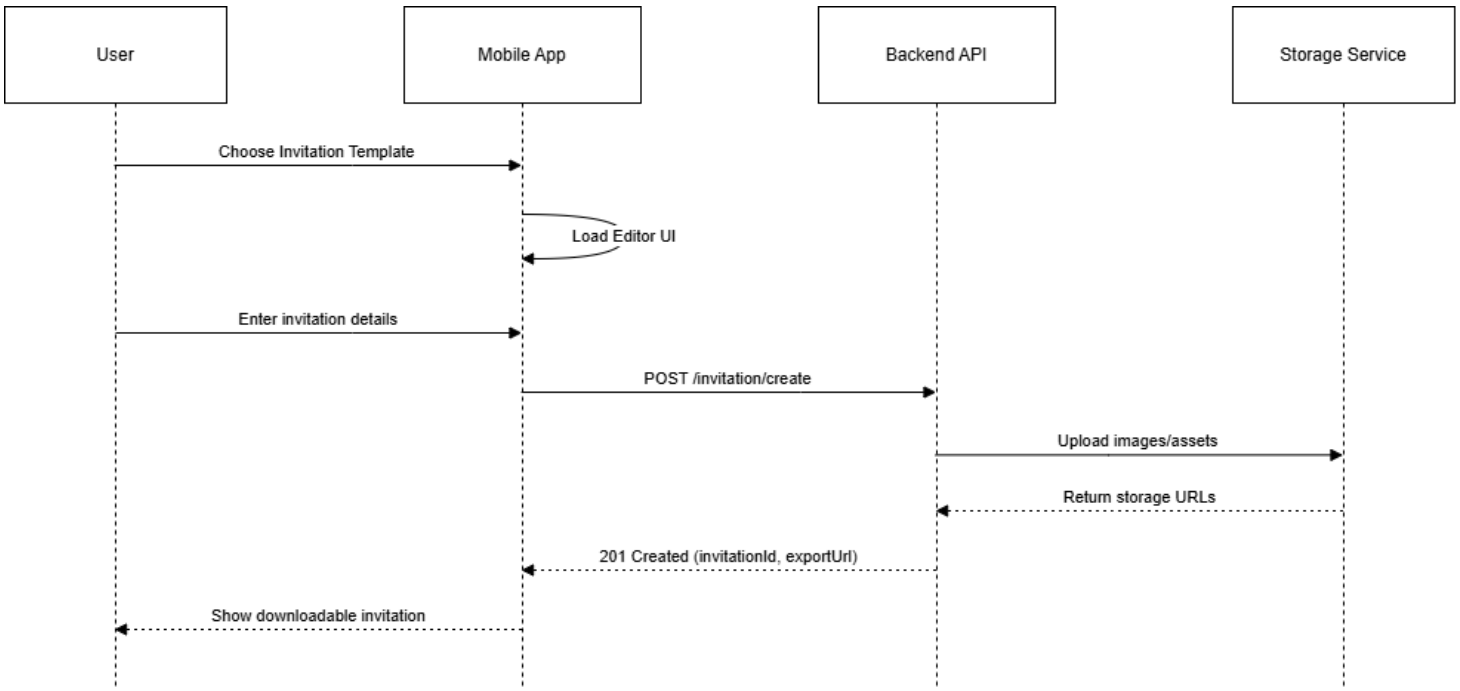


Figure 3.2.1.5 - Sequence Diagram – Create Digital Invitation

3.2.1.6 Sequence Diagram – Chatbot Conversation

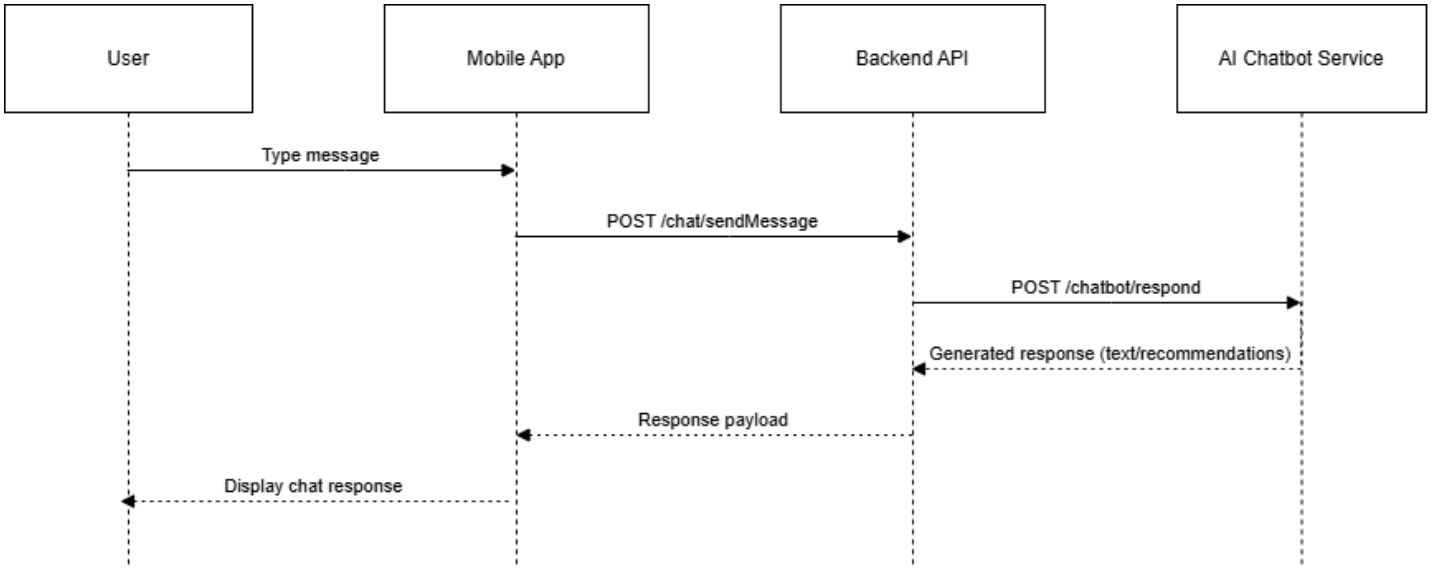


Figure 3.2.1.6 - Sequence Diagram – Chatbot Conversation

3.2.2 Activity Diagrams

3.2.2.1 Activity Diagram – User Registration Workflow

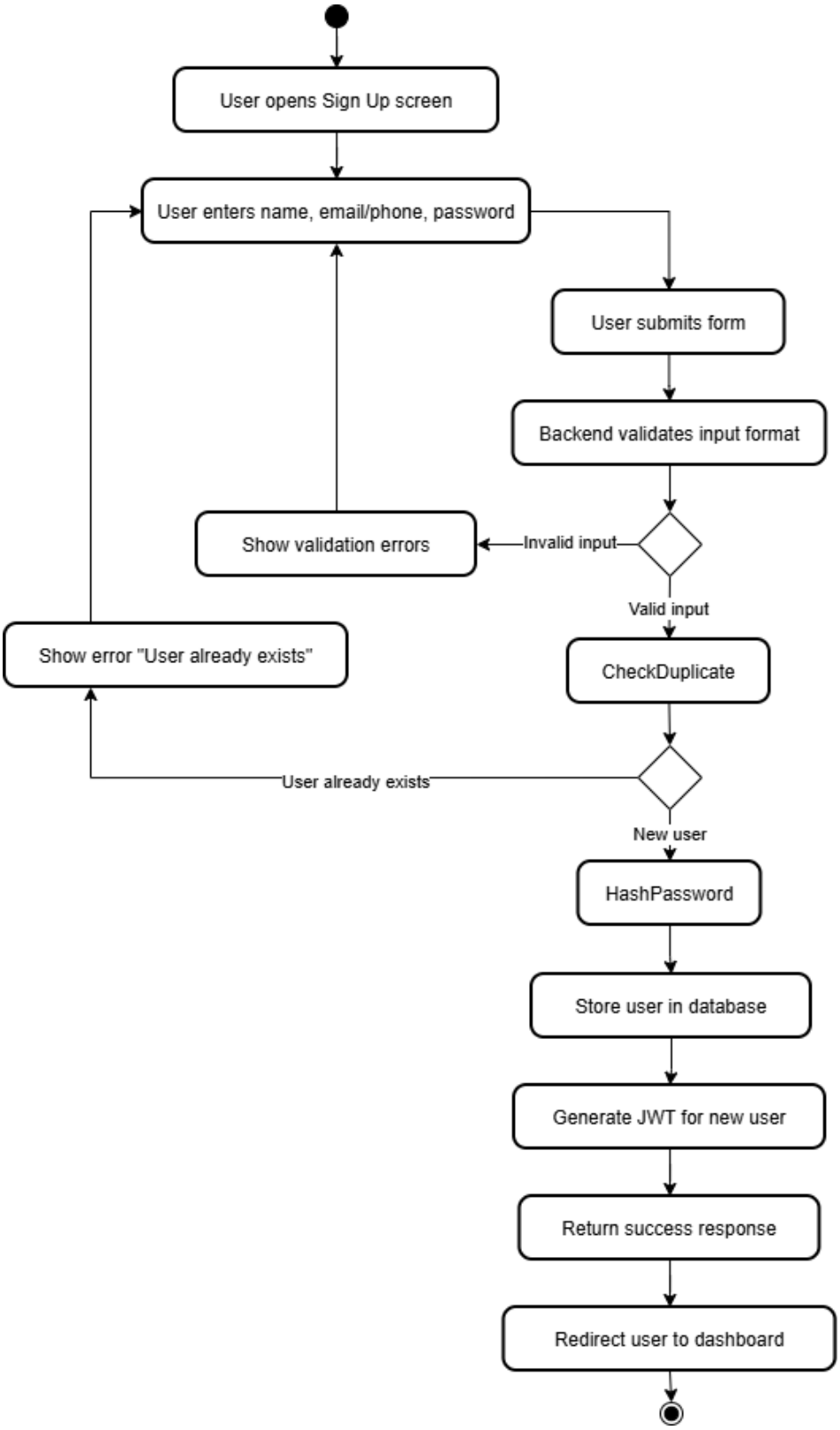


Figure 3.2.2.1 - Activity Diagram – User Registration Workflow

3.2.2.2 Activity Diagram – Invitation Creation

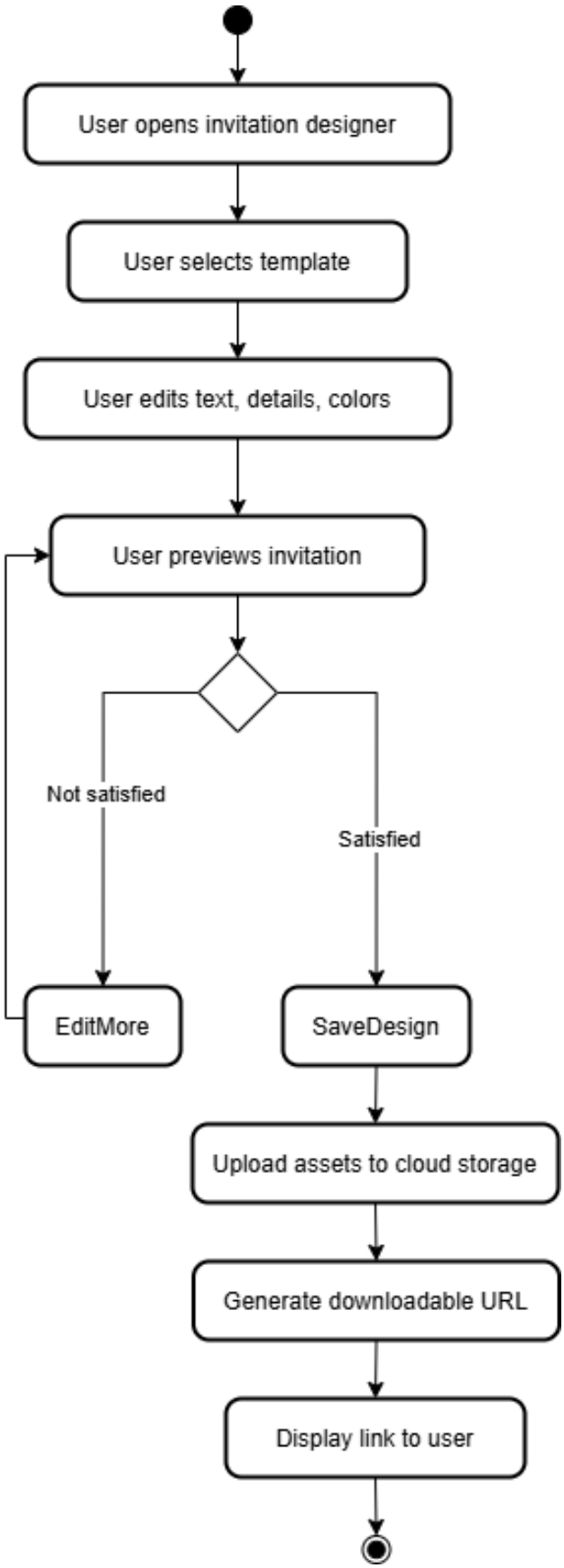


Figure 3.2.2.2 - Activity Diagram – Invitation Creation

3.2.2.3 Activity Diagram – Vendor Booking

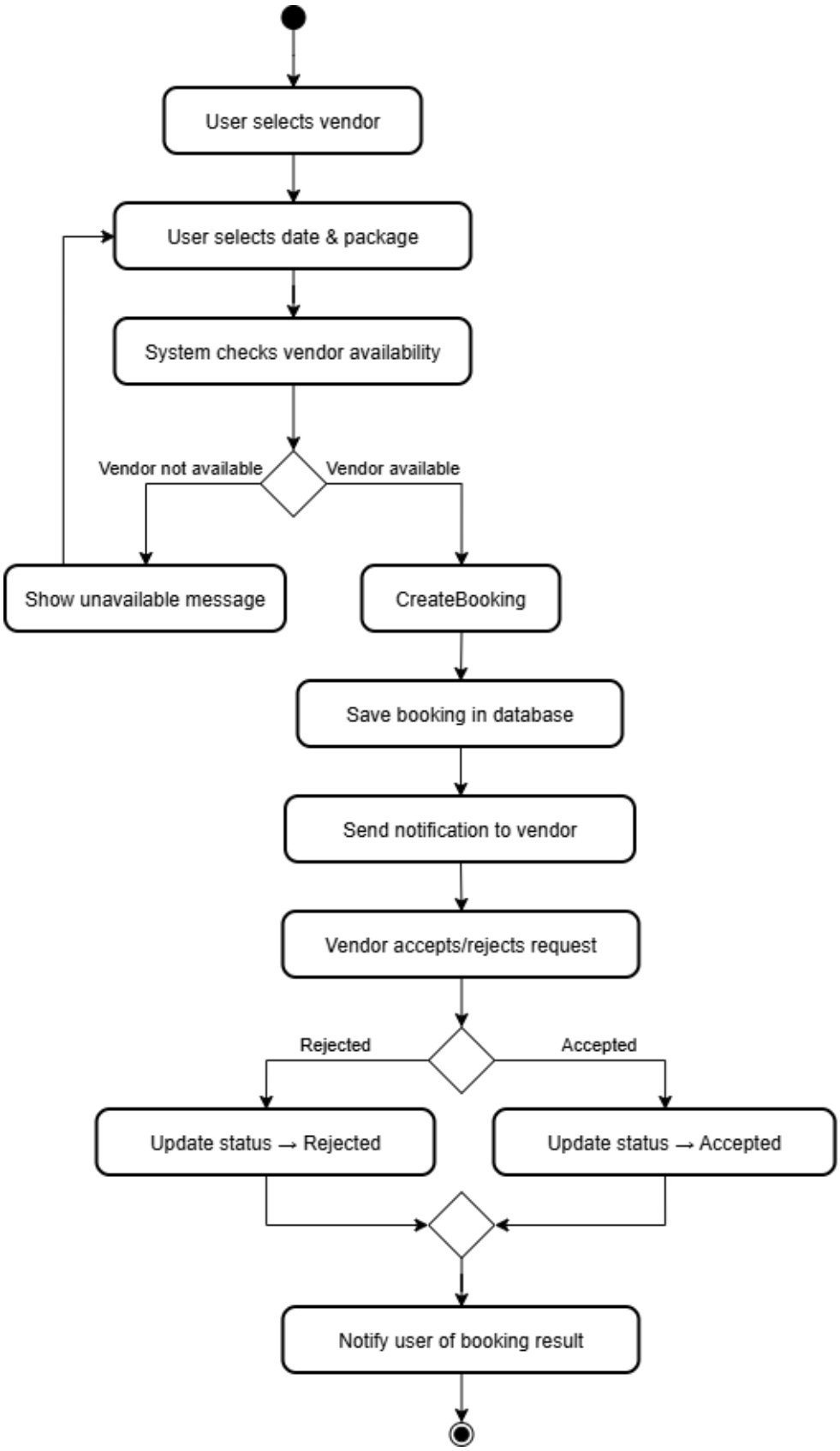


Figure 3.2.2.3 - Activity Diagram – Vendor Booking

3.3 Deployment View

3.3.1 Deployment Diagram

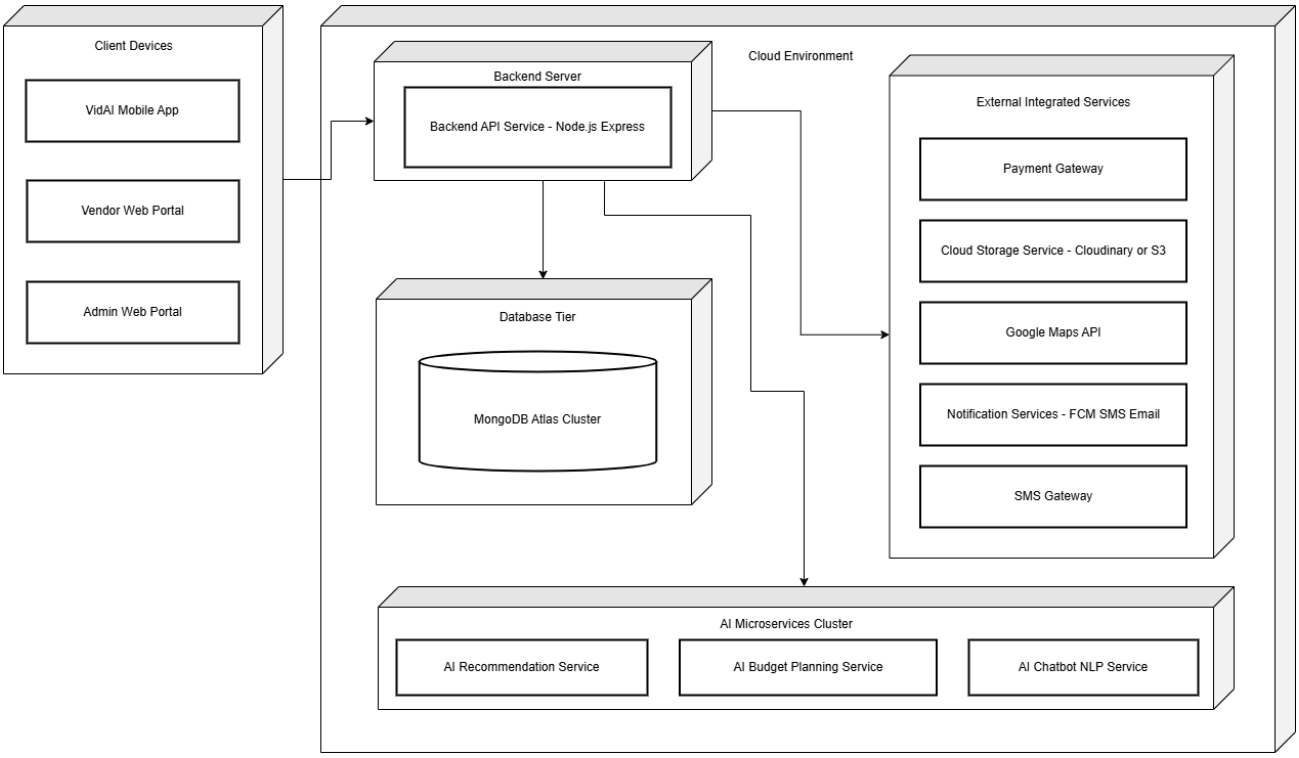


Figure 3.3.1 - Deployment Diagram

3.4 Development View

3.4.1 Component Diagram

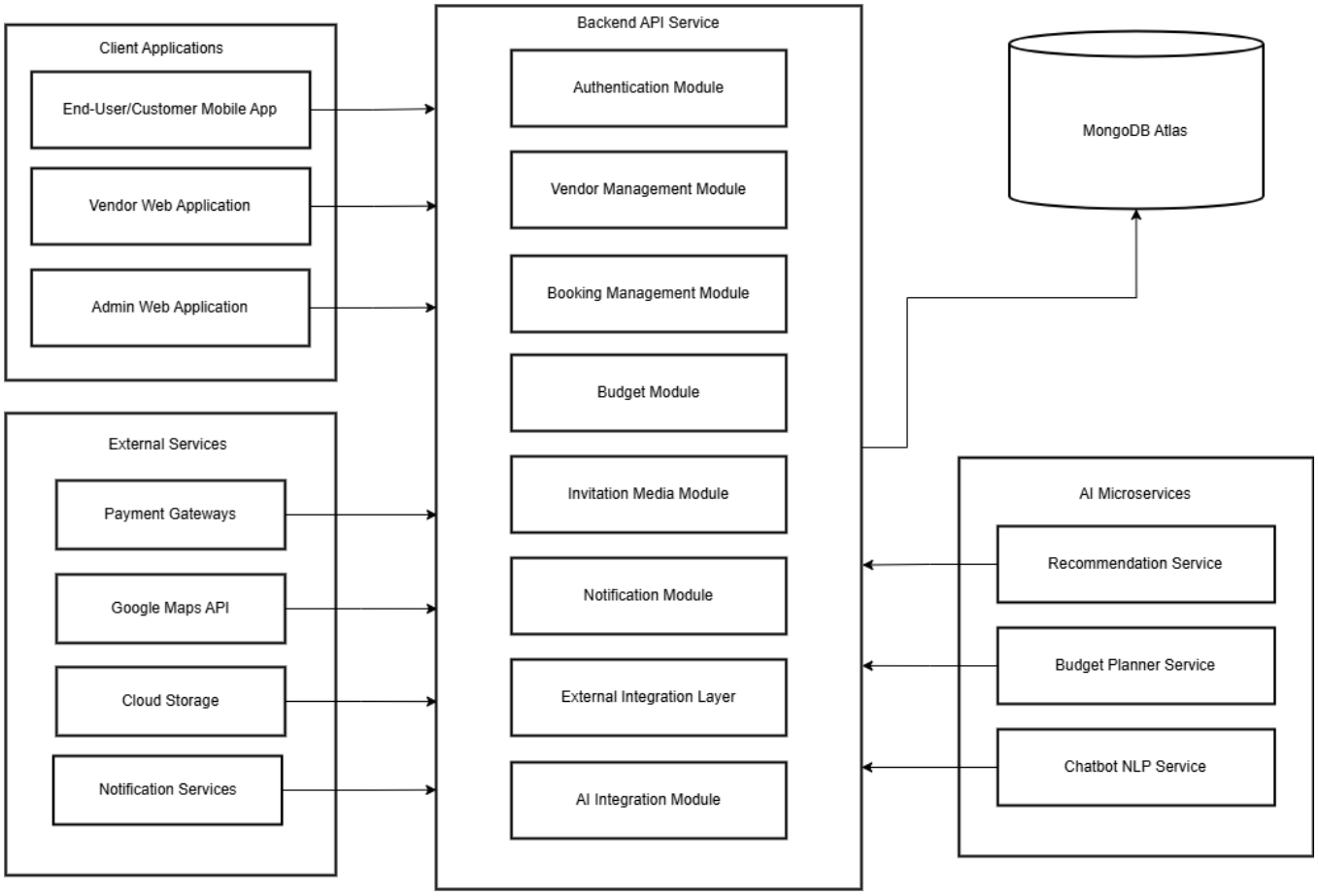
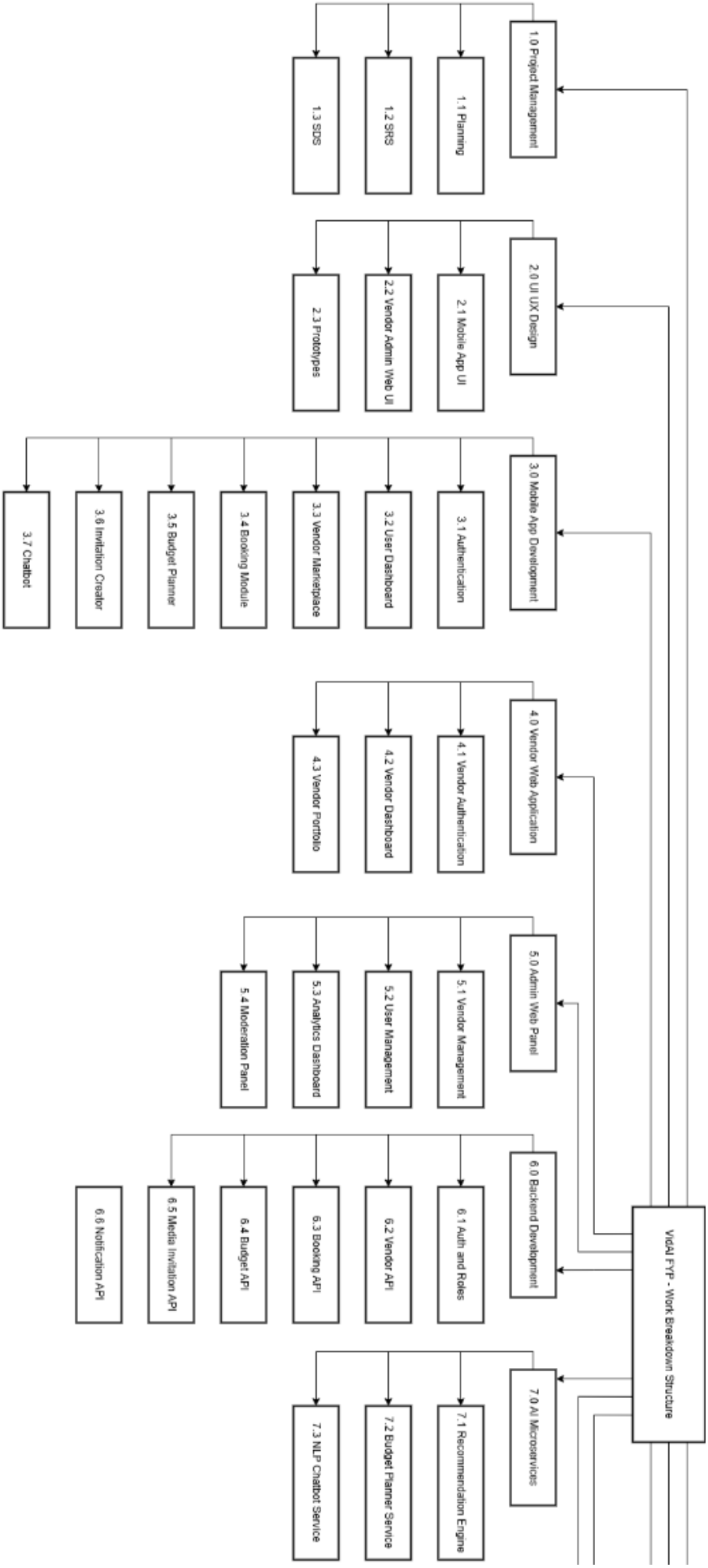


Figure 3.4.1 - Component Diagram

3.4.2 WBS


VidAI FYP - Work Breakdown Structure
1.0 Project Management
1.1 Planning
1.2 SRS
1.3 SDS
2.0 UI UX Design
2.1 Mobile App UI
2.2 Vendor Admin Web UI
2.3 Prototypes
3.0 Mobile App Development
3.1 Authentication
3.2 User Dashboard
3.3 Vendor Marketplace
3.4 Booking Module
3.5 Budget Planner
3.6 Invitation Creator
3.7 Chatbot
4.0 Vendor Web Application
4.1 Vendor Authentication
4.2 Vendor Dashboard
4.3 Vendor Portfolio
5.0 Admin Web Panel
5.1 Vendor Management
5.2 User Management
5.3 Analytics Dashboard
5.4 Moderation Panel
6.0 Backend Development
6.1 Auth and Roles
6.2 Vendor API
6.3 Booking API
6.4 Budget API
6.5 Media Invitation API
6.6 Notification API
7.0 AI Microservices
7.1 Recommendation Engine
7.2 Budget Planner Service
7.3 NLP Chatbot Service

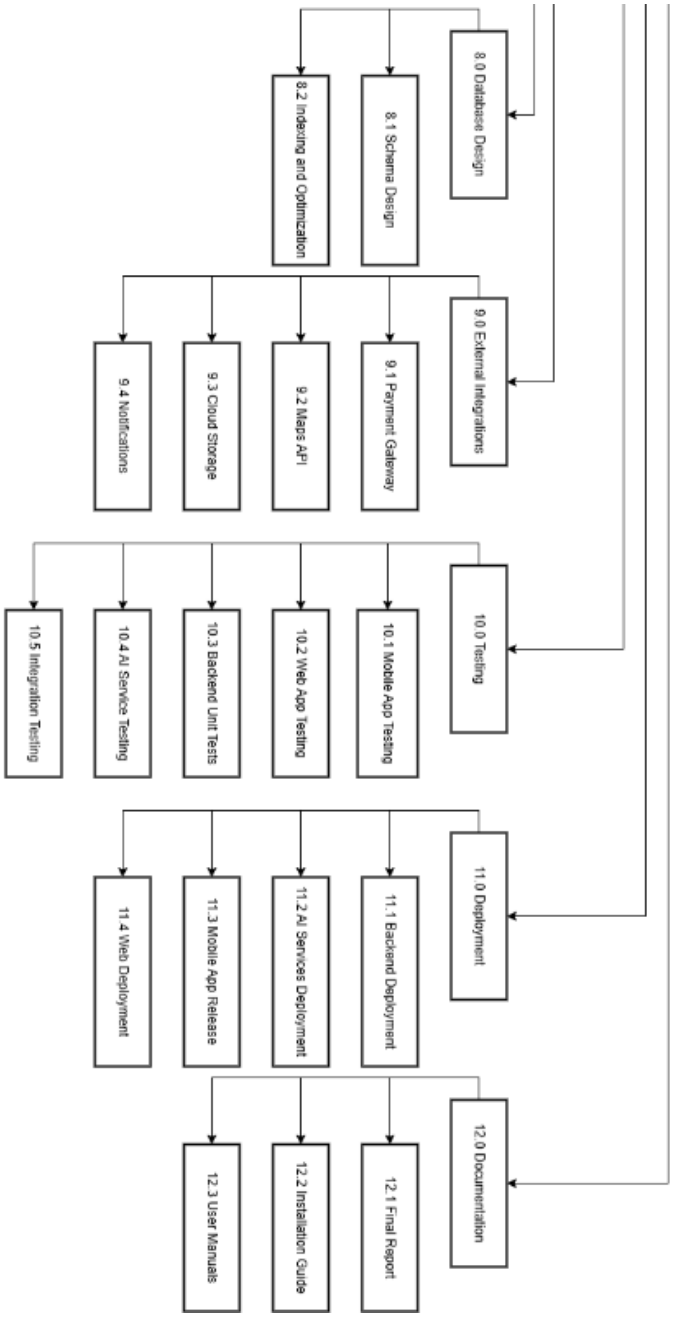


Figure 3.4.2 – WBS

3.5 Data Model

3.5.1 ER Diagram

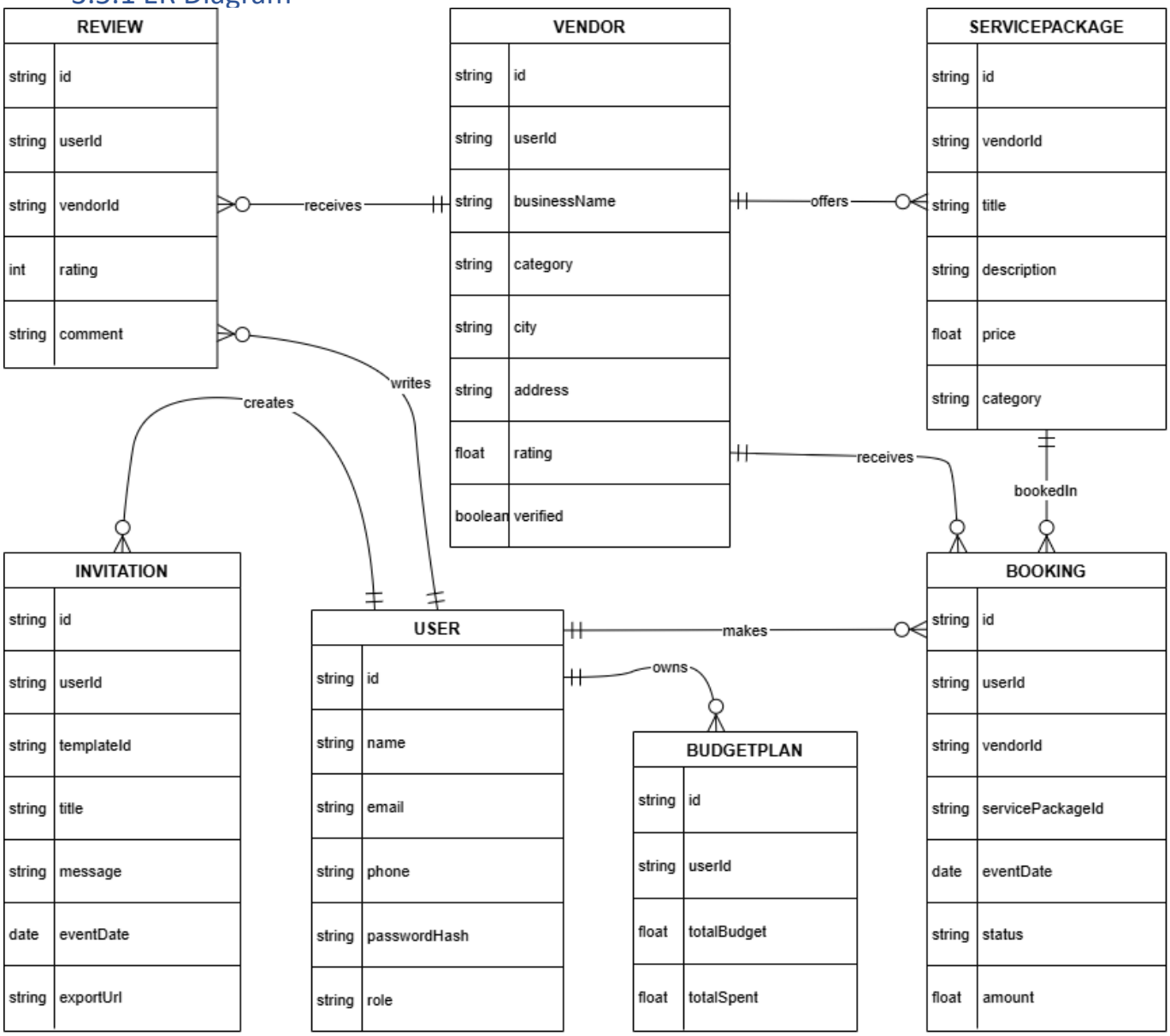


Figure 3.5.1 - ER Diagram

3.6 User Interface Design

This section describes the logical UI designs, screen layouts, and major interface elements for the VidAI system.

The descriptions below follow the updated architecture, where:

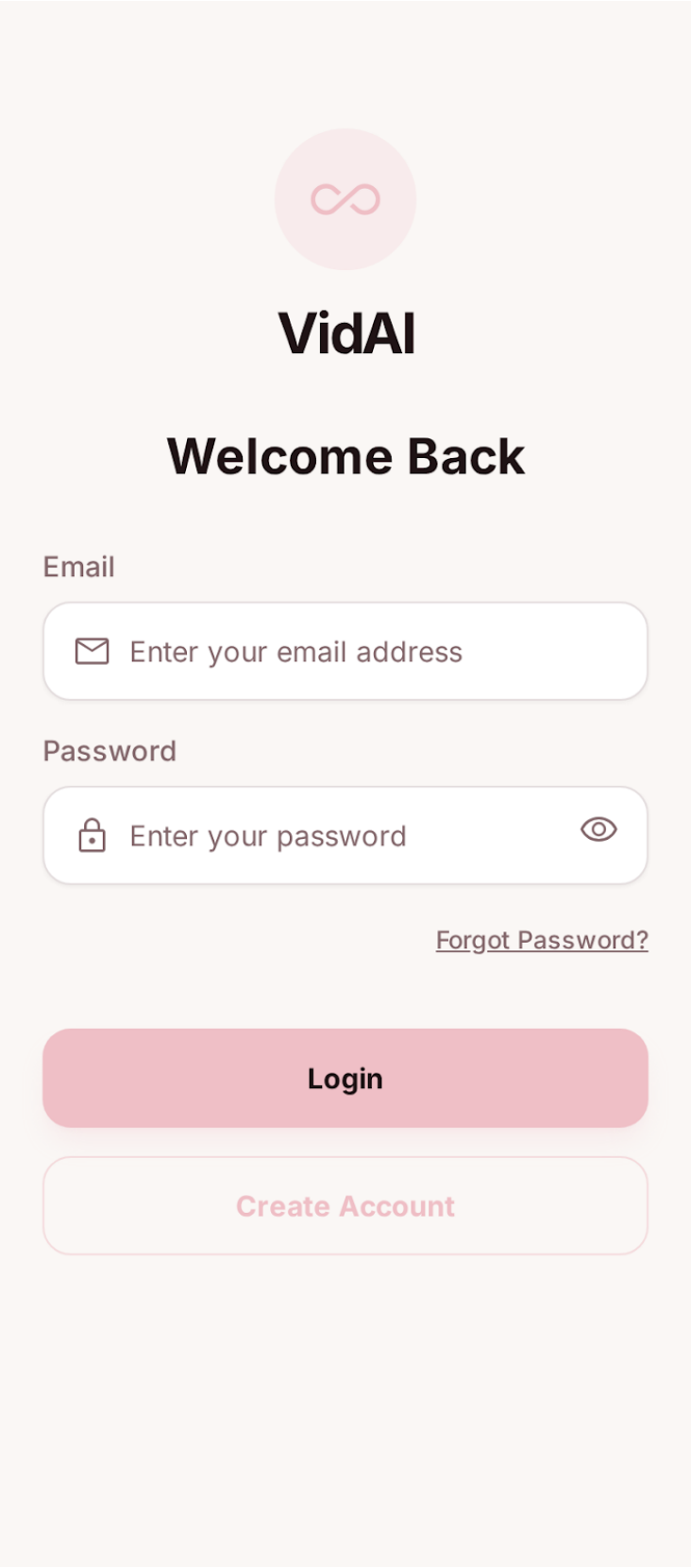
- **React Native Mobile App** → End Users (Bride/Groom/Family)
- **React Web App** → Vendors & Admins

All UI components adhere to Material Design guidelines, mobile responsiveness, and accessibility requirements.

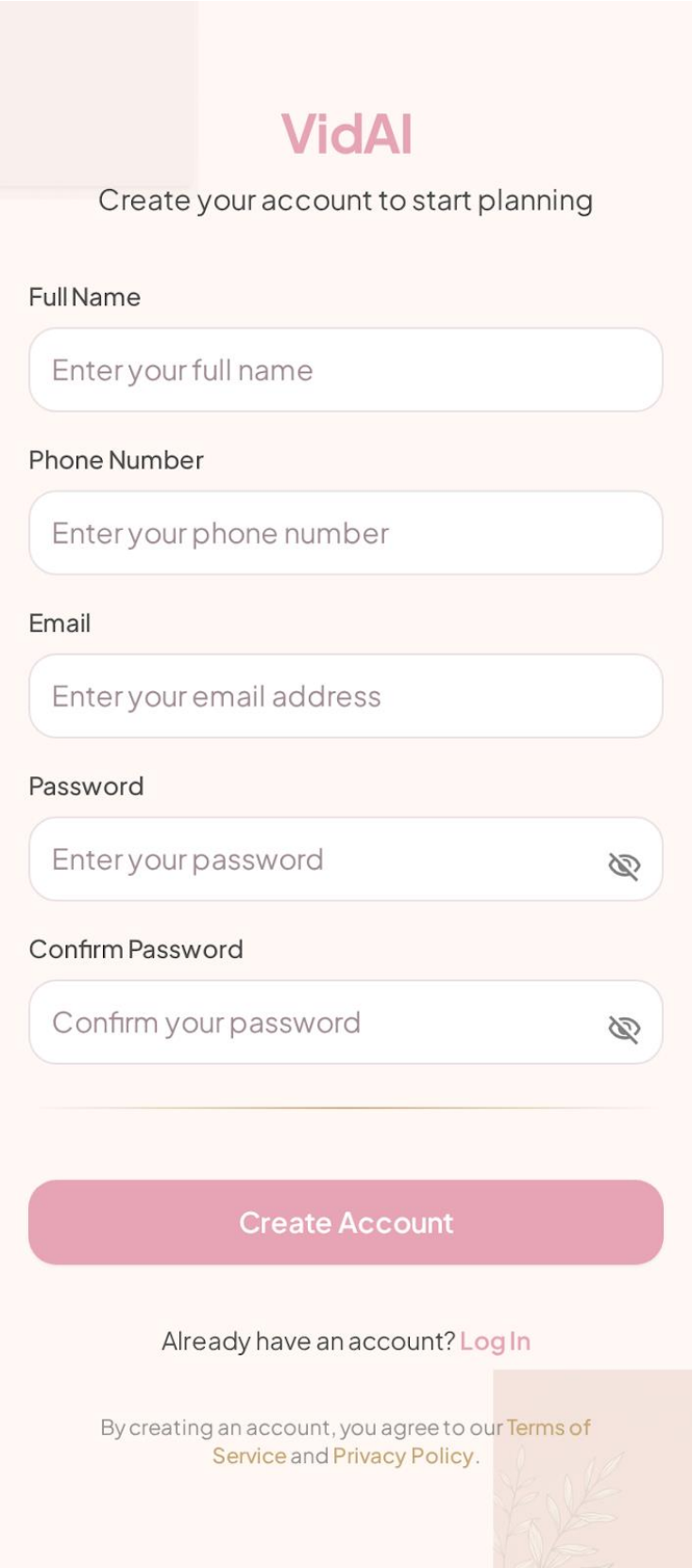
3.6.1 Mobile Application

3.6.1.1 Welcome Screen

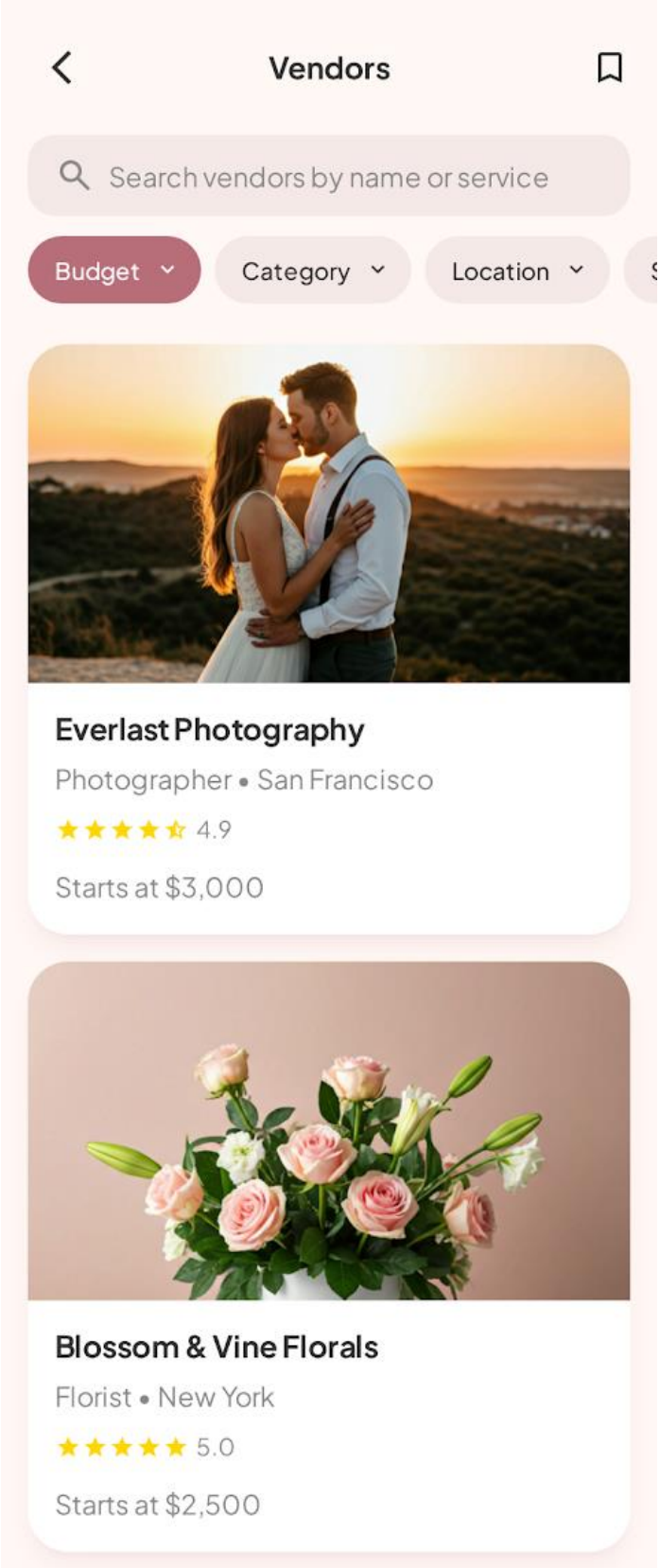
3.6.1.2 Customer Login Screen



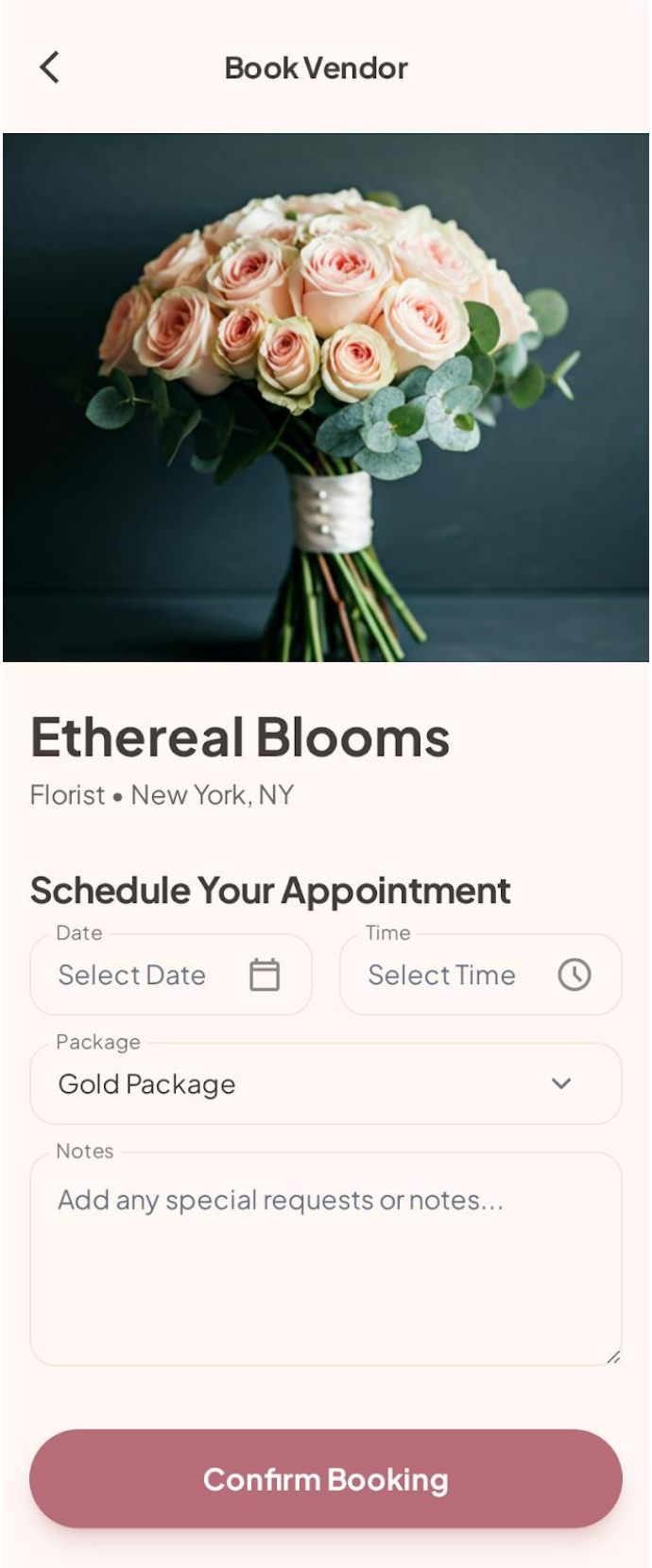
3.6.1.3 Customer Sign Up Screen



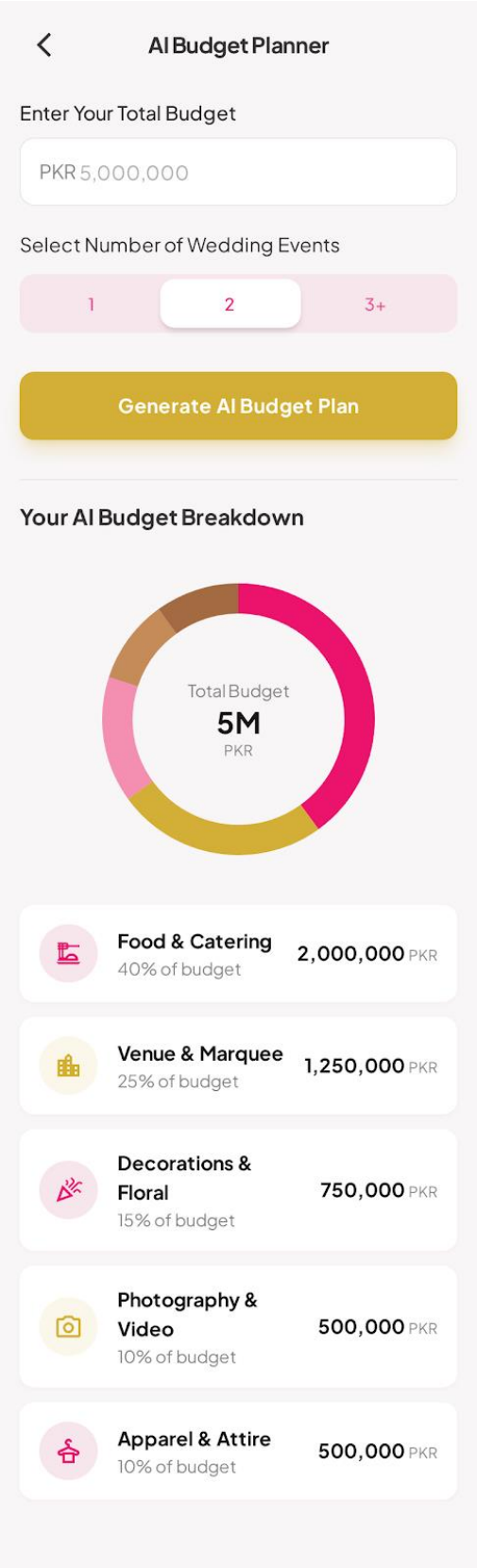
3.6.1.4 Vendor Listings



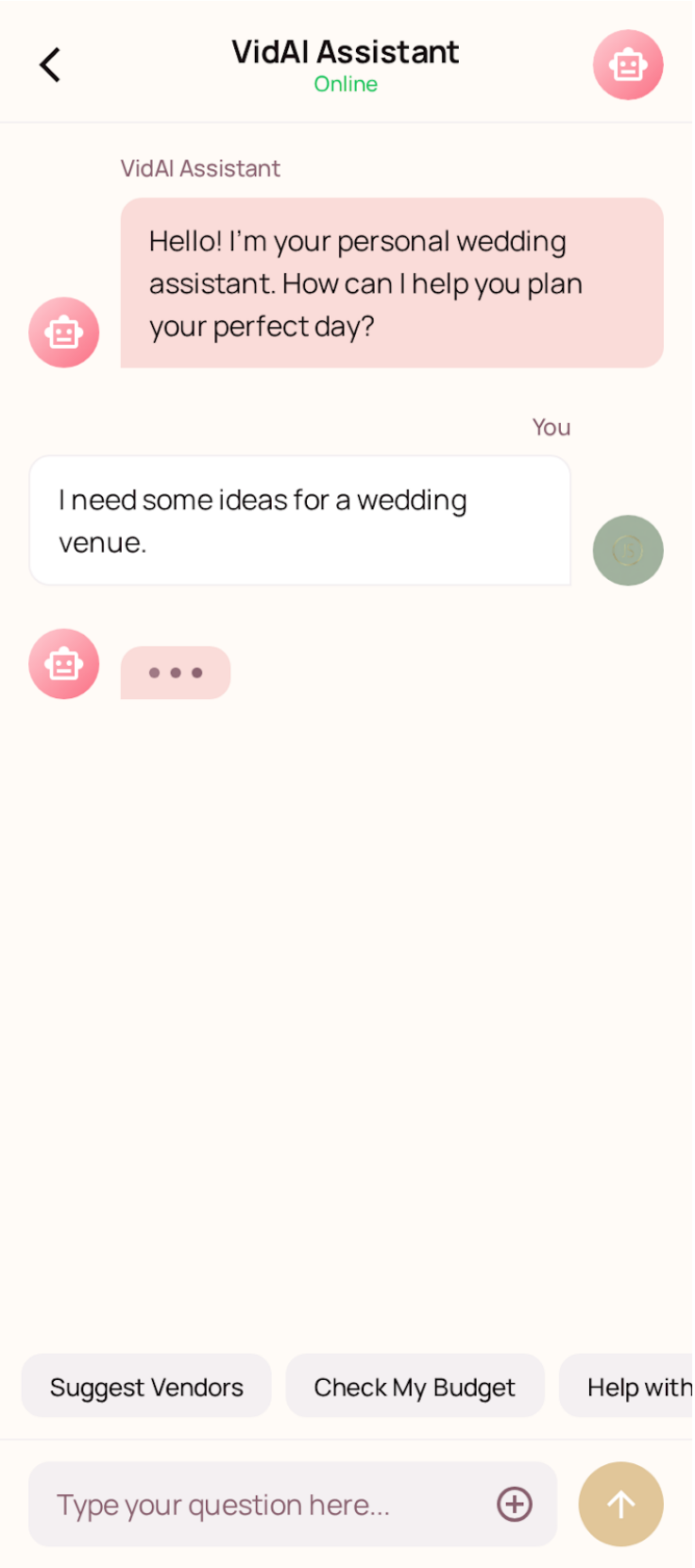
3.6.1.5 Vendor Booking Screen



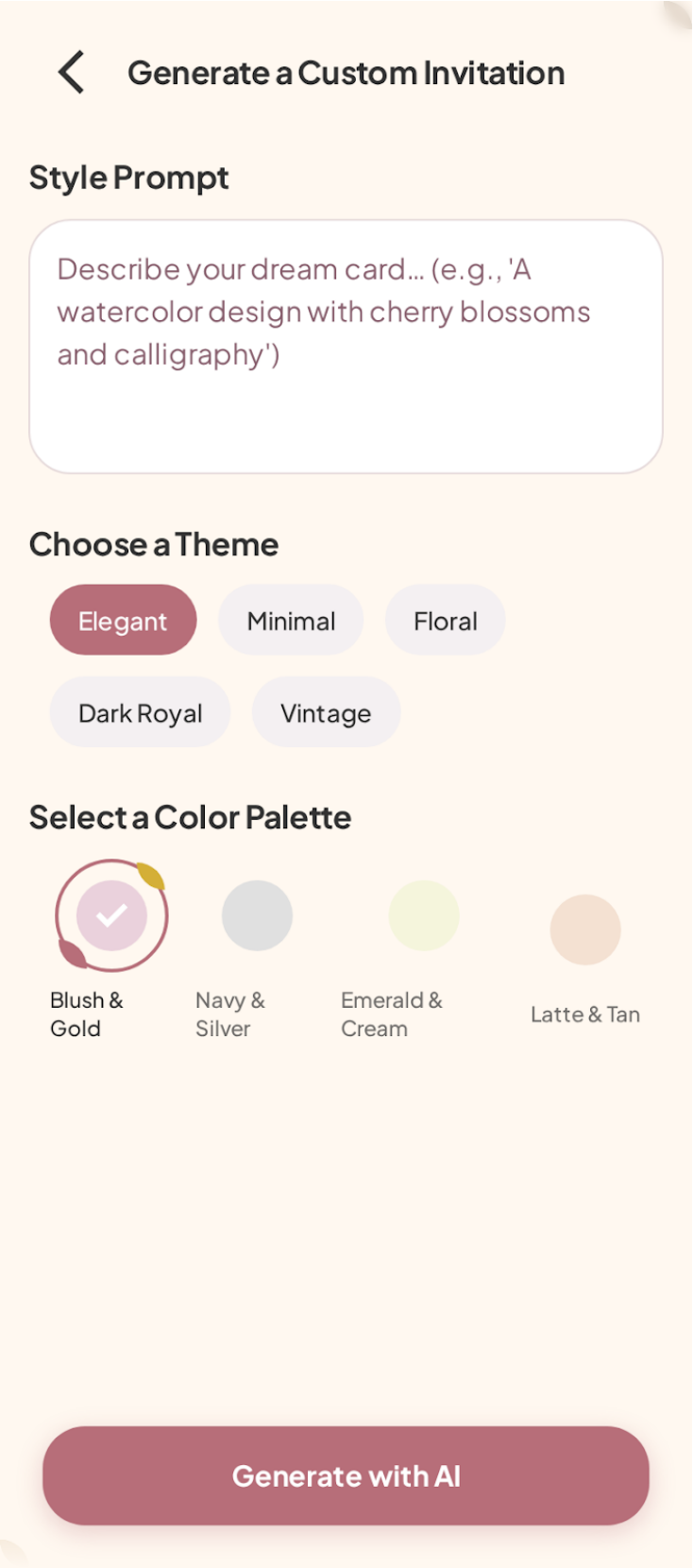
3.6.1.6 Budget Planner



3.6.1.7 Chatbot Screen

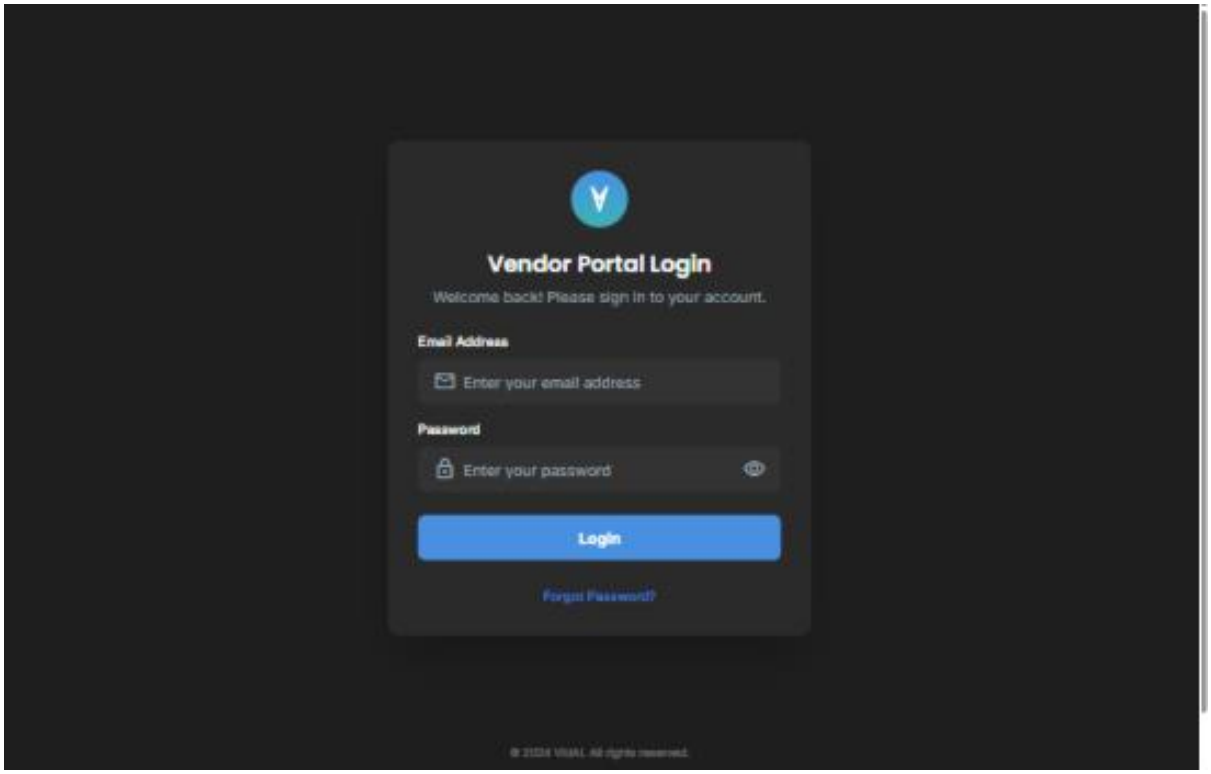


3.6.1.8 Digital Invitation Generation

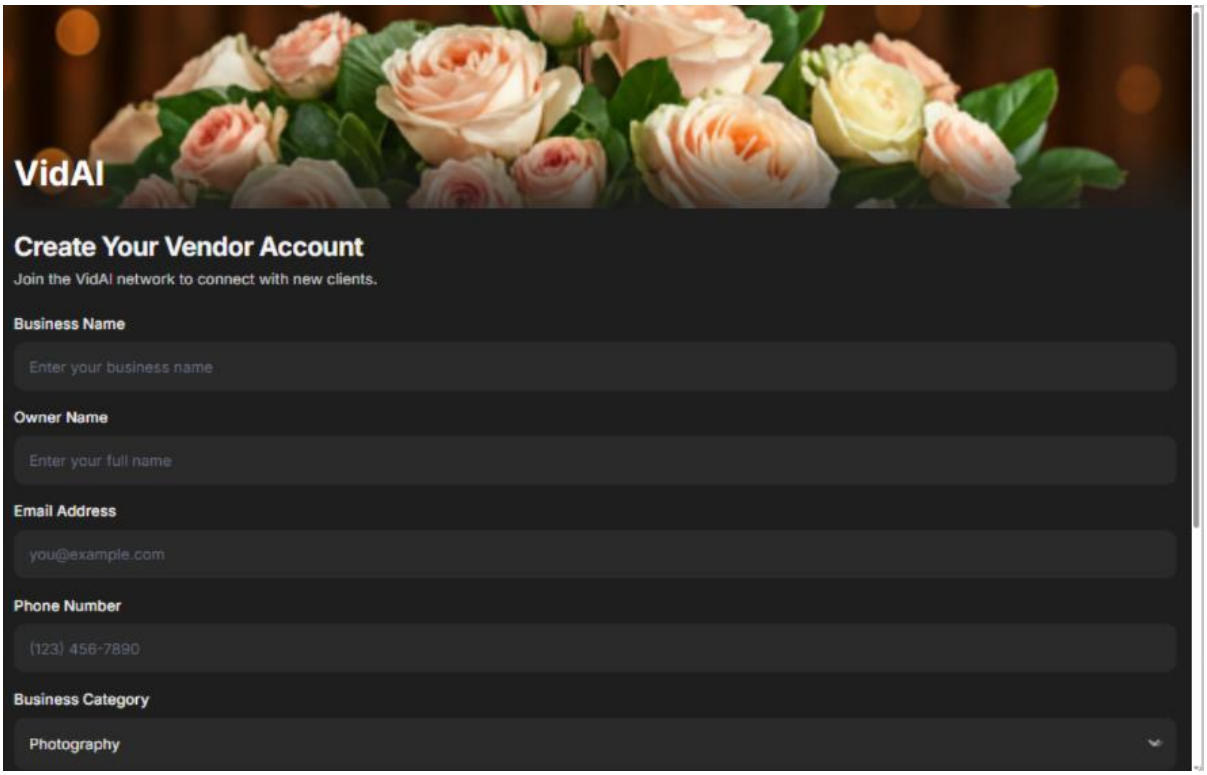


3.6.2 Vendor Web Portal

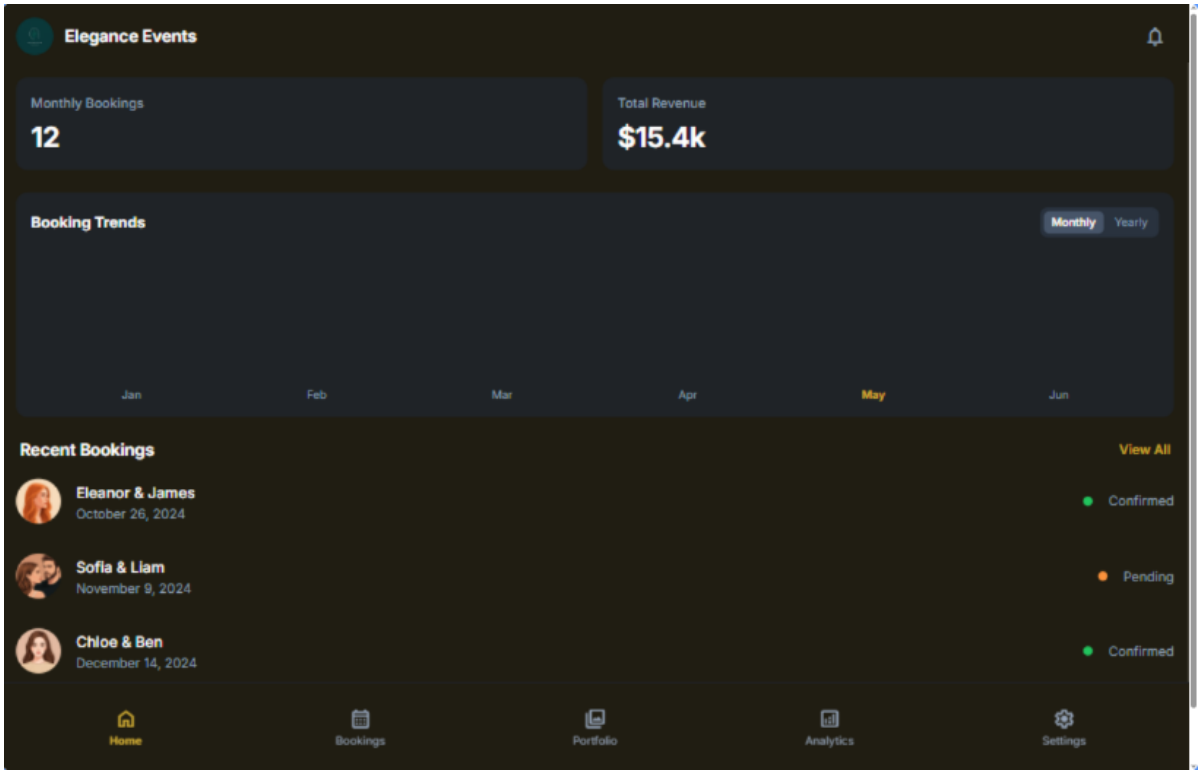
3.6.2.1 Vendor Login Screen



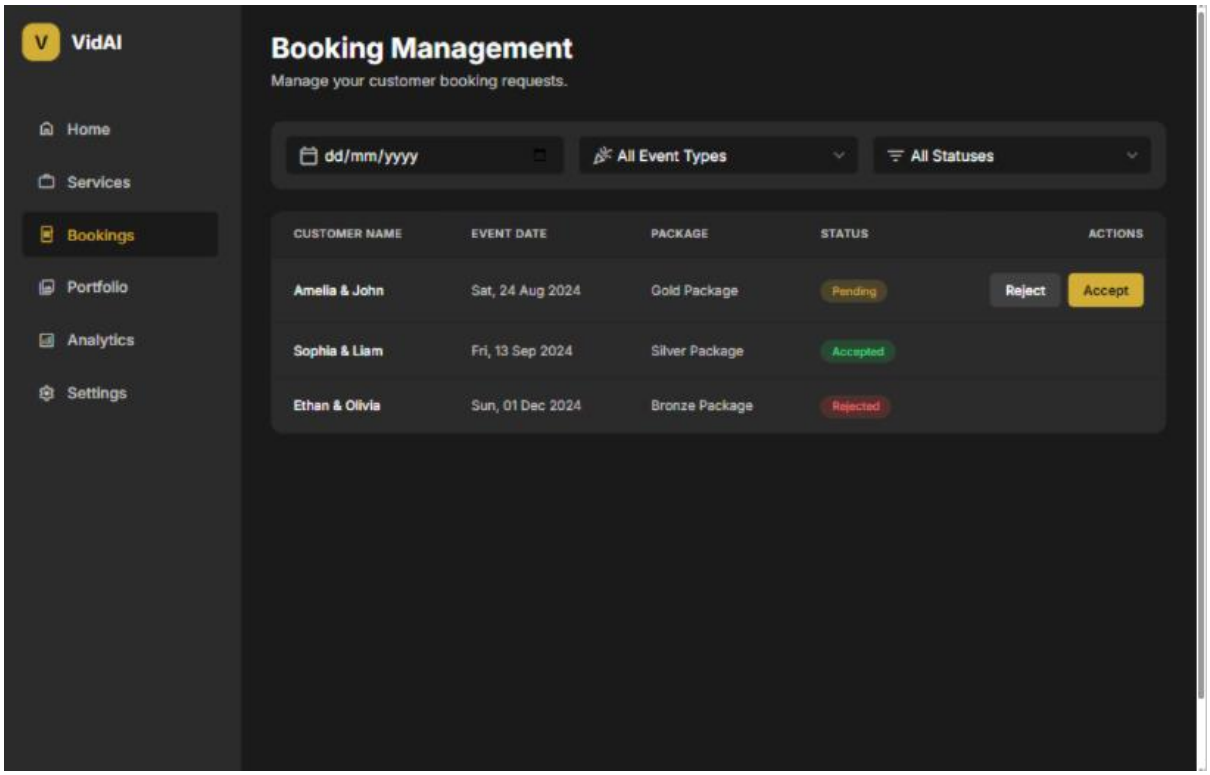
3.6.2.2 Vendor Registration Screen



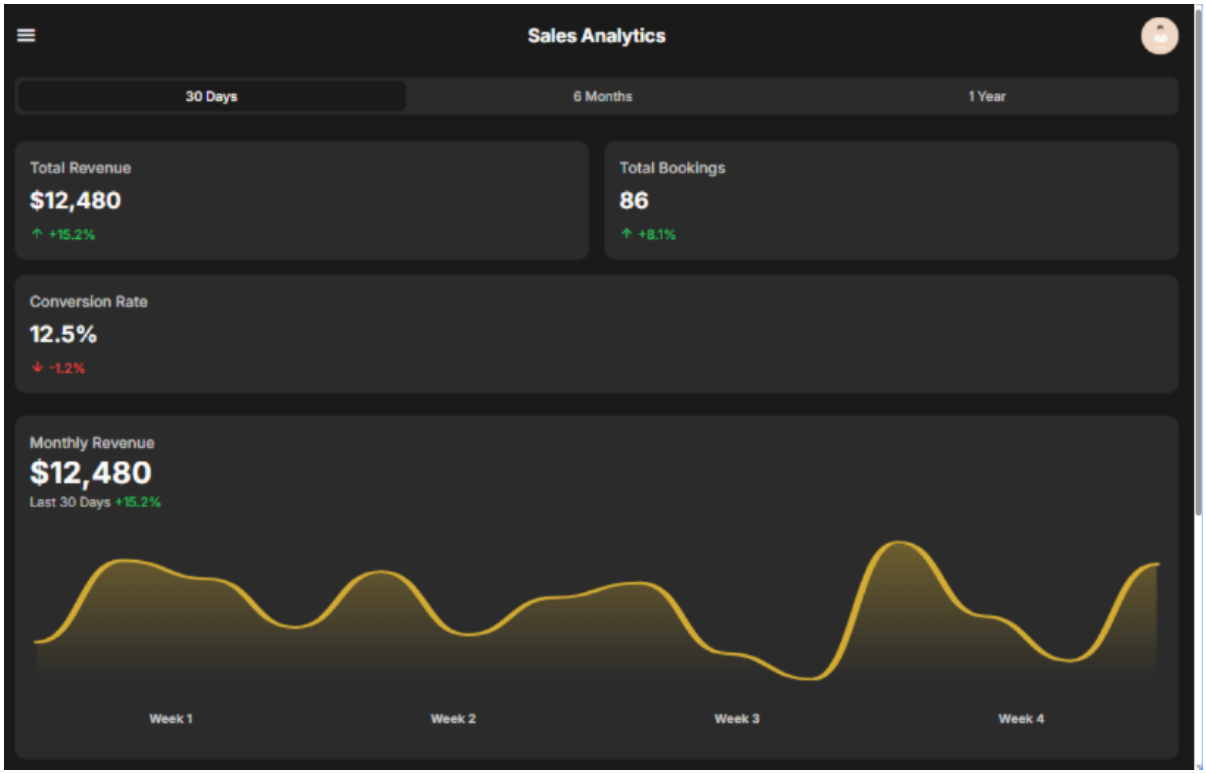
3.6.2.3 Vendor Dashboard



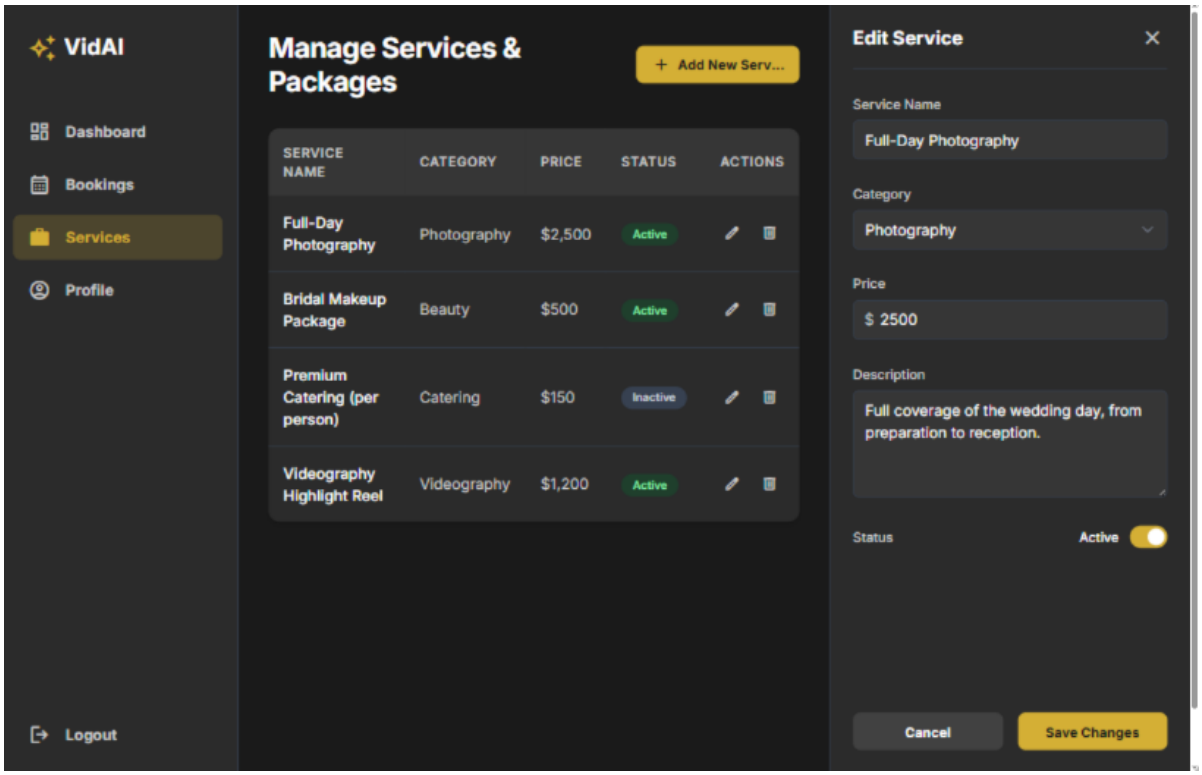
3.6.2.4 Booking Management



3.6.2.5 Sales Analytics

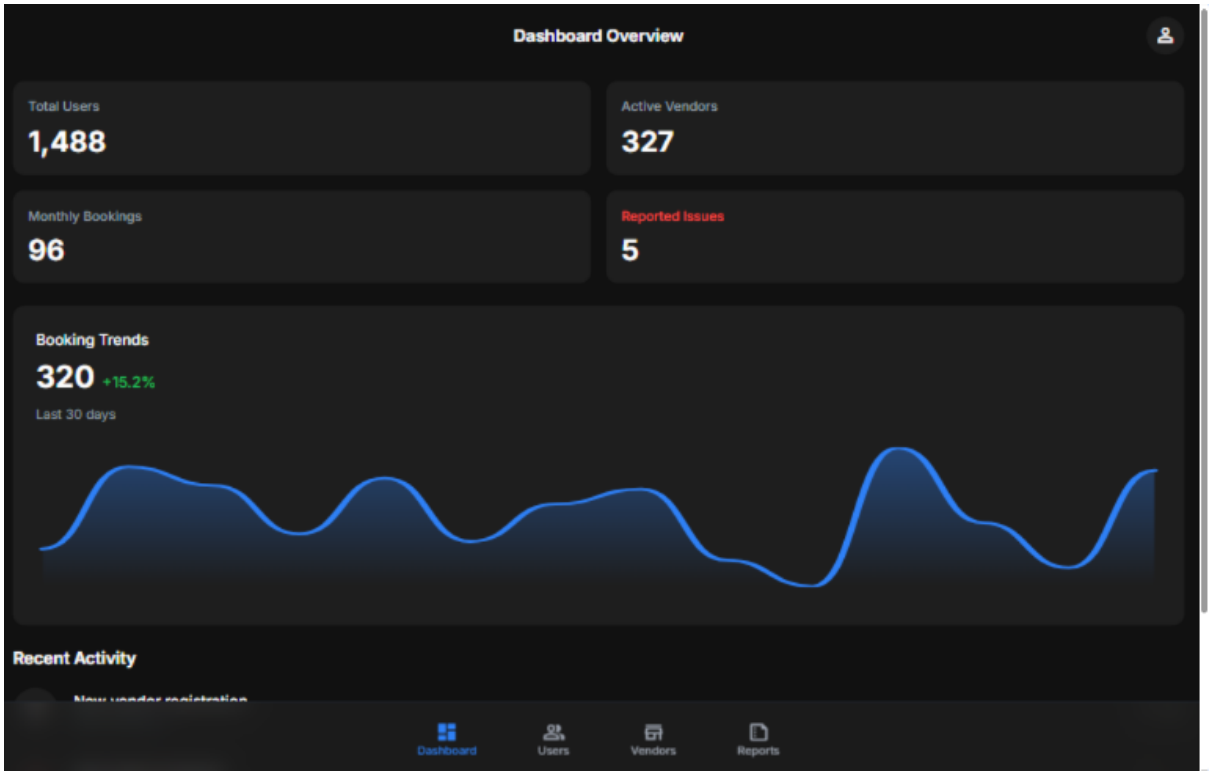


3.6.2.6 Manage Services/Packages

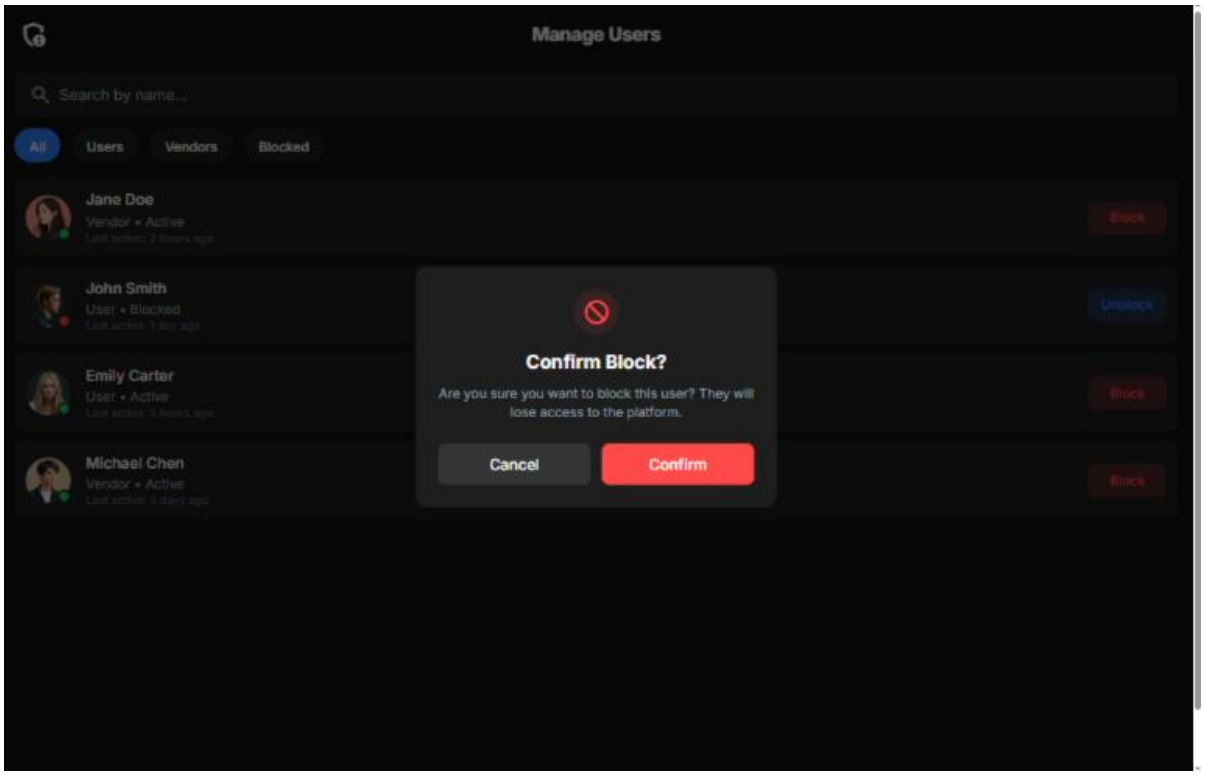
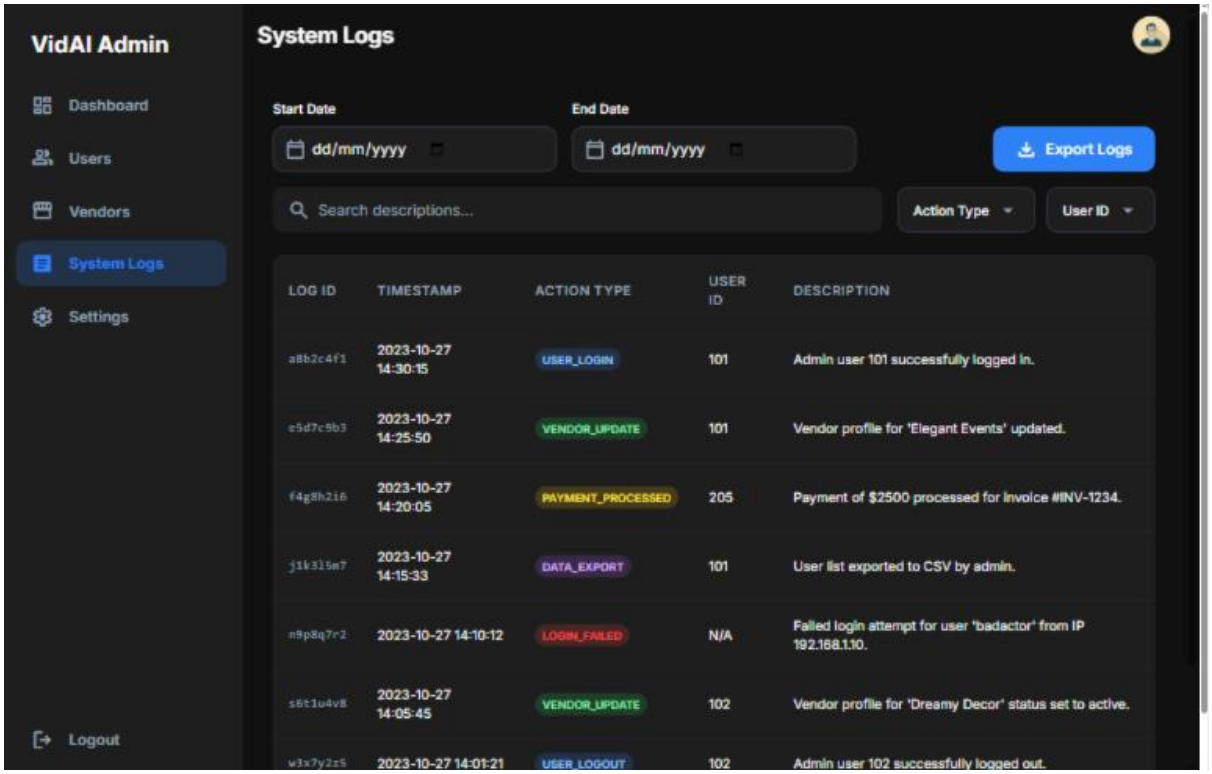


3.6.3 Admin Web Portal

3.6.3.1 Admin Dashboard



3.6.3.2 Manage Users



3.6.3.3 System Logs